第2章 80x86微处理器

本章教学内容

- 80X86 CPU的基本组成和寄存器作用
- 标志寄存器各位的含义
- 实模式下存储器的分段、20位物理地址的形成
- 段地址寄存器和偏移地址寄存器之间的隐含规则

主要内容

2.1 80x86内部结构与内部寄存器组

2.2 80x86的引脚信号

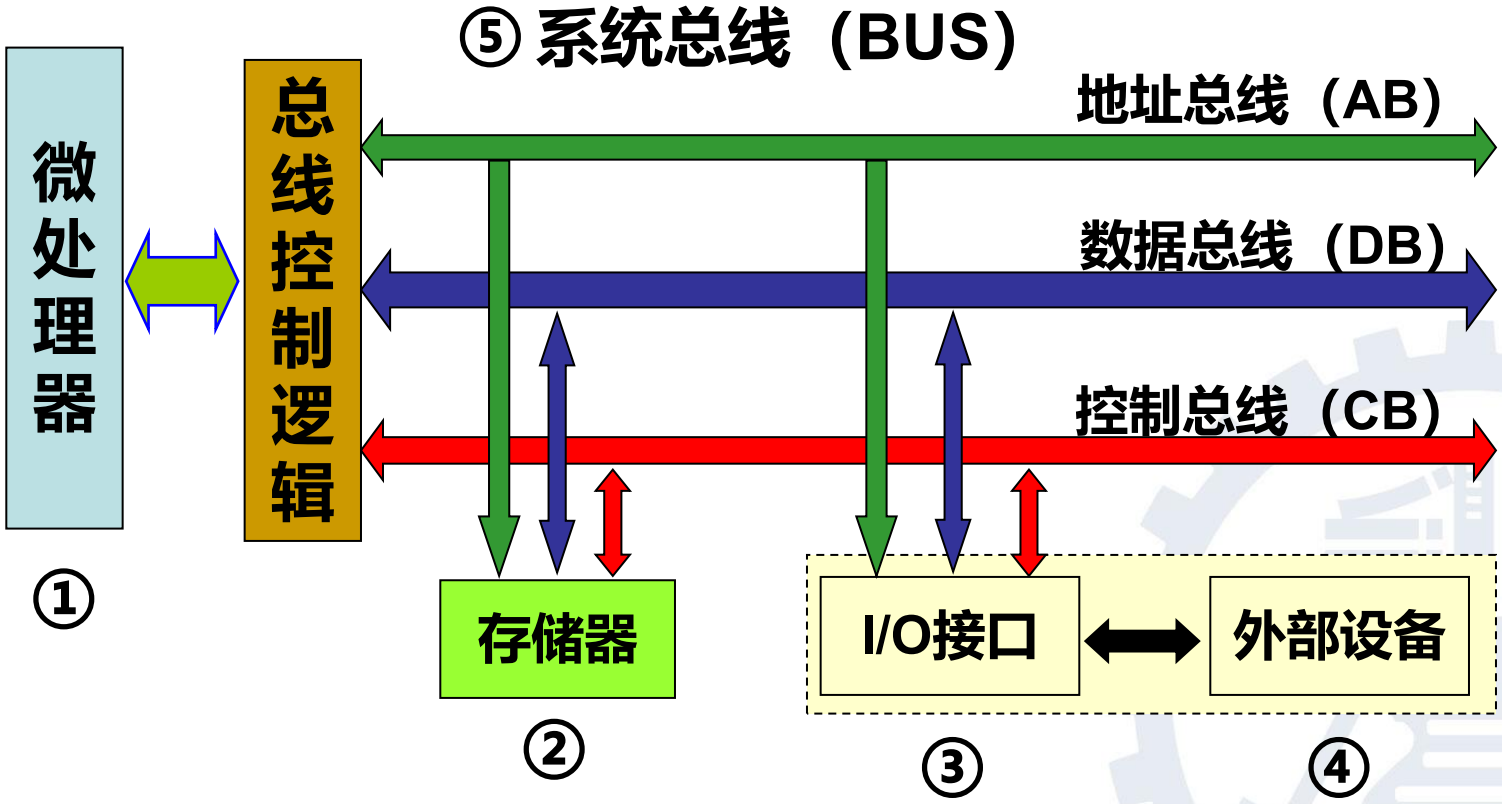
2.3 80x86系统总线的构成

2.4 80x86的工作方式

2.1 80x86内部结构与内部寄存器组

计算机系统

以16位的8086为教学对象



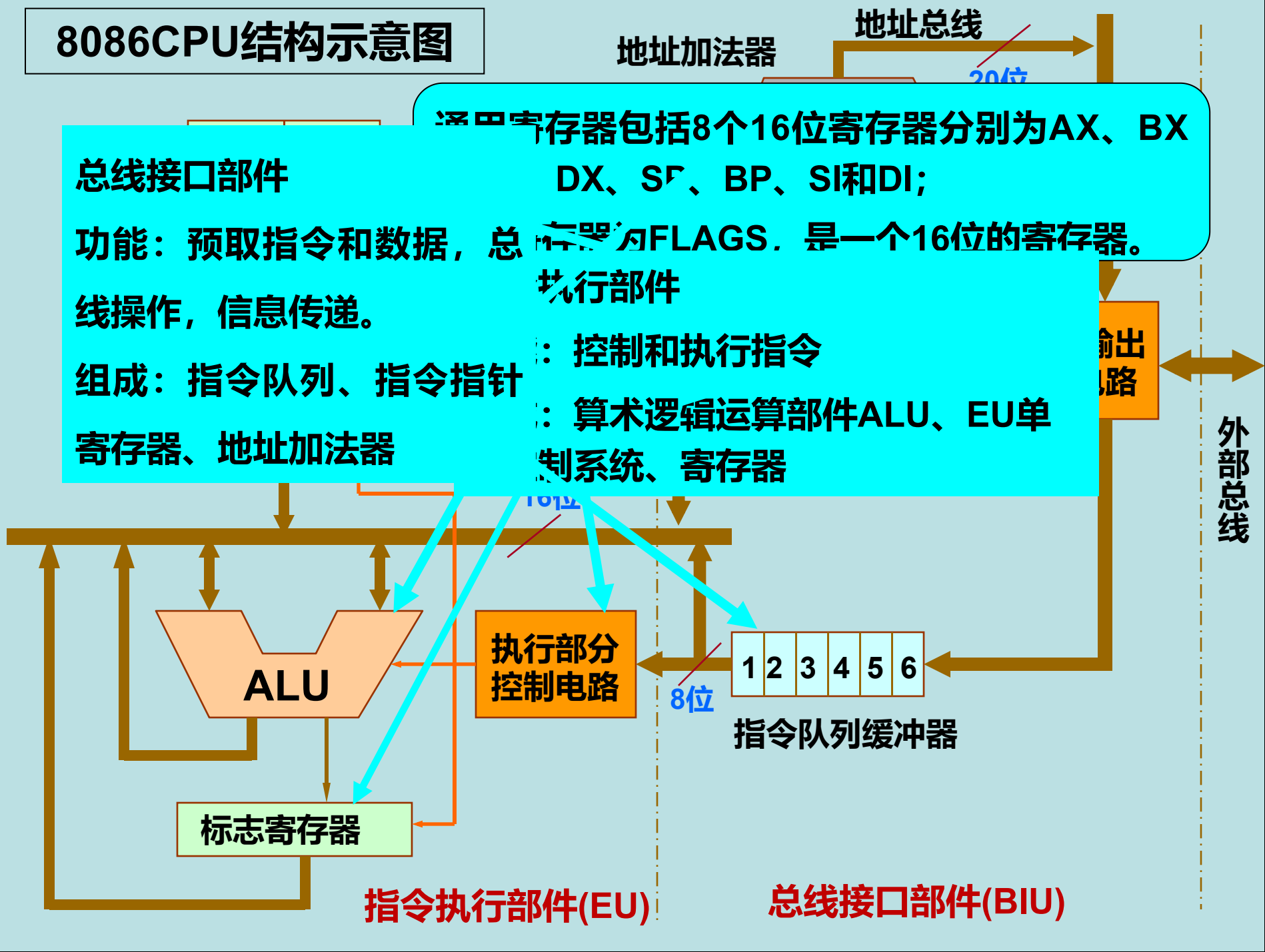
1. 8086 的基本性能指标

- 16位微处理器
- 16根数据线、20根地址线，可寻址的地址空间达1MB (2^{20}Byte = 1MB)
- 可以和浮点运算器、I/O处理器或其他处理器组成多处理器系统，从而提高系统的数据吞吐能力和数据处理能力。

2. 8086 的功能结构

- 指令执行部件(EU)
  - 算术逻辑运算器 (ALU)
  - 状态标志寄存器 (FLAGS)
  - 暂存寄存器
  - 通用寄存器组
  - EU控制单元
- 总线接口部件(BIU)
  - 指令队列缓冲器
  - 16位指令指针和段寄存器IP
  - 地址产生器和段寄存器
  - 总线控制逻辑

8086CPU结构示意图



- 总线接口部件
  - 负责管理与系统总线的接口，负责对存储器和外设访问
  - 包括指令队列、指令指针、段寄存器、地址加法器和总线控制逻辑
- 指令执行部件
  - 包括ALU、通用寄存器、标志寄存器和控制电路
  - 负责指令译码、数据运算和指令执行
- 指令执行的两个主要阶段：取指和执行
  - 取指：从主存取出指令代码进入指令队列
  - 执行：译码指令、并发出有关控制信号实现指令功能

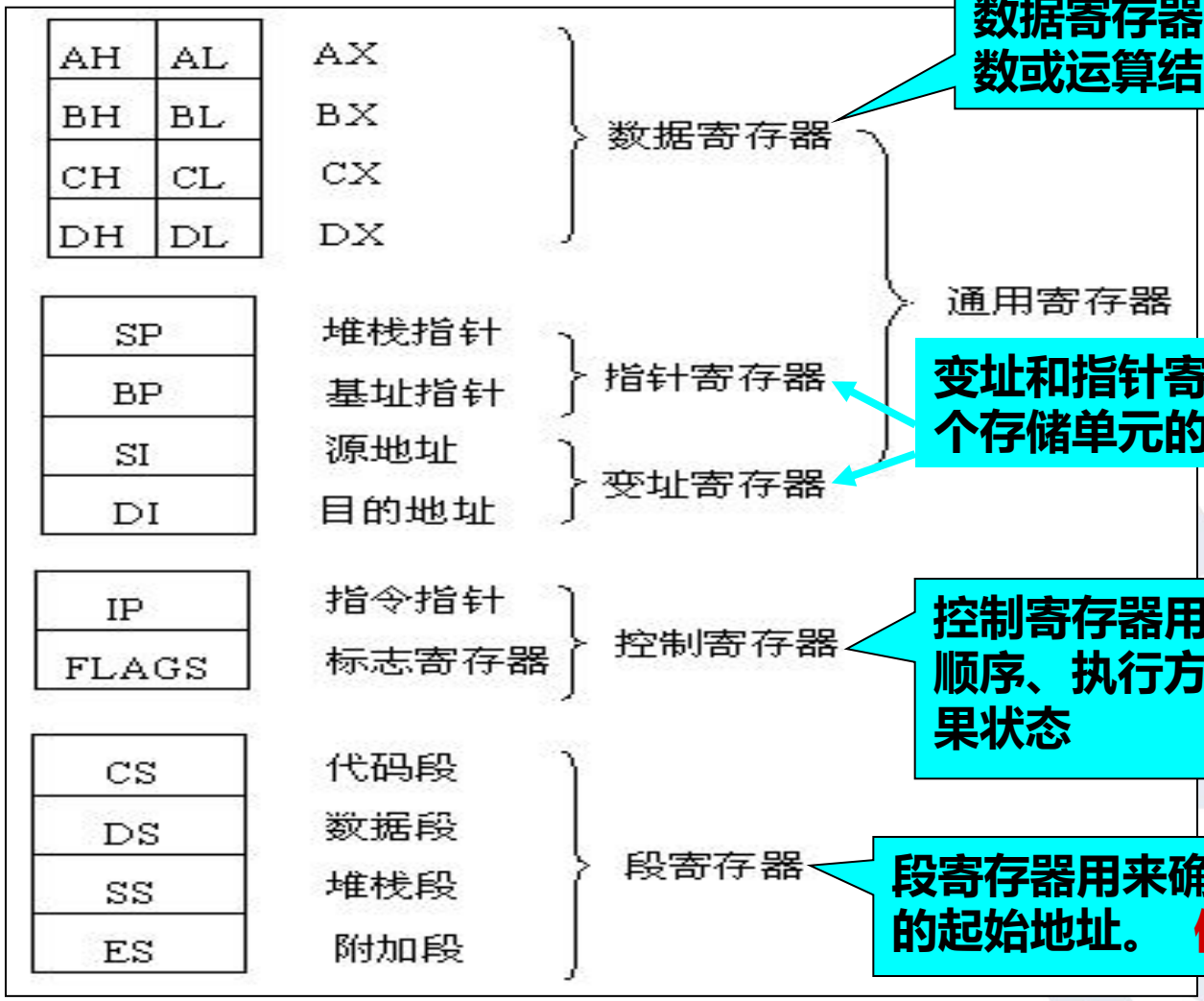


指令预取（Prefetch）

- 8086处理器的指令读取，实际上是指令预取
  - 8086处理器维护着长度为6个字节的指令队列
  - EU单元译码、执行指令，同时BIU单元读取后续指令
  - BIU和EU两个单元相互独立，可以并行操作
- 最简单的指令流水线技术
  - 节省许多取指时间，提高了工作效率



3. 8086的寄存器结构



数据寄存器用来保存操作数或运算结果等

变址和指针寄存器用于存放某个存储单元的偏移地址

控制寄存器用来控制指令执行顺序、执行方式，保存运行结果状态

段寄存器用来确定该段在内存中的起始地址。 便于OS执行程序


AH
AL
BH
BL
CH
CL
DH
DL
SP
BP
SI
DI
IP
FLAGS
CS
DS
SS
ES
数据段
堆栈段
附加段
段寄存器
数据寄存器用来保存操作数或运算结果等
SP用于存放当前堆栈段中栈顶的偏移地址；BP用于存放堆栈段中
在字符串操作中，SI和DI都具有
保存下一次将要取出指令的偏移地址，IP的内
8086CPU的 4 个16位的段寄存器:CS，DS，SS，ES。
80386以后增添了FS和GS。
段寄存器用来确定该段在内存中的起始地址。
控制标志
功能:控制处理器执行指
（1）方向标志DF
（2）中断允许标志IF
（3）追踪标志TF

通用寄存器的一般用途

寄存器	用　　法
AX	累加器，用于算术、逻辑运算以及与外设传送信息等
BX	基址寄存器,常于存放存储器地址
CX	计数器，作为循环或串操作等指令中的隐含计数器
DX	数据寄存器,用来存放双字数据的高16位，或存放外设端口地址
BP	BP用于存放堆栈段中某一存储单元的偏移地址
SP	堆栈指针，用于存放当前堆栈段中栈顶的偏移地址
SI	串操作中的源地址指针，具有自动增量或减量的功能
DI	串操作中的目的地址指针，具有自动增量或减量的功能
IP	保存下一条将要执行指令的偏移地址。的内容由微处理器硬件自动设置，有一些指令可改变IP的值，如转移指令、子程序调用指令等
FLAGS	包含9个标志位，保存一条指令执行后CPU所处状态信息及运算结果的特征

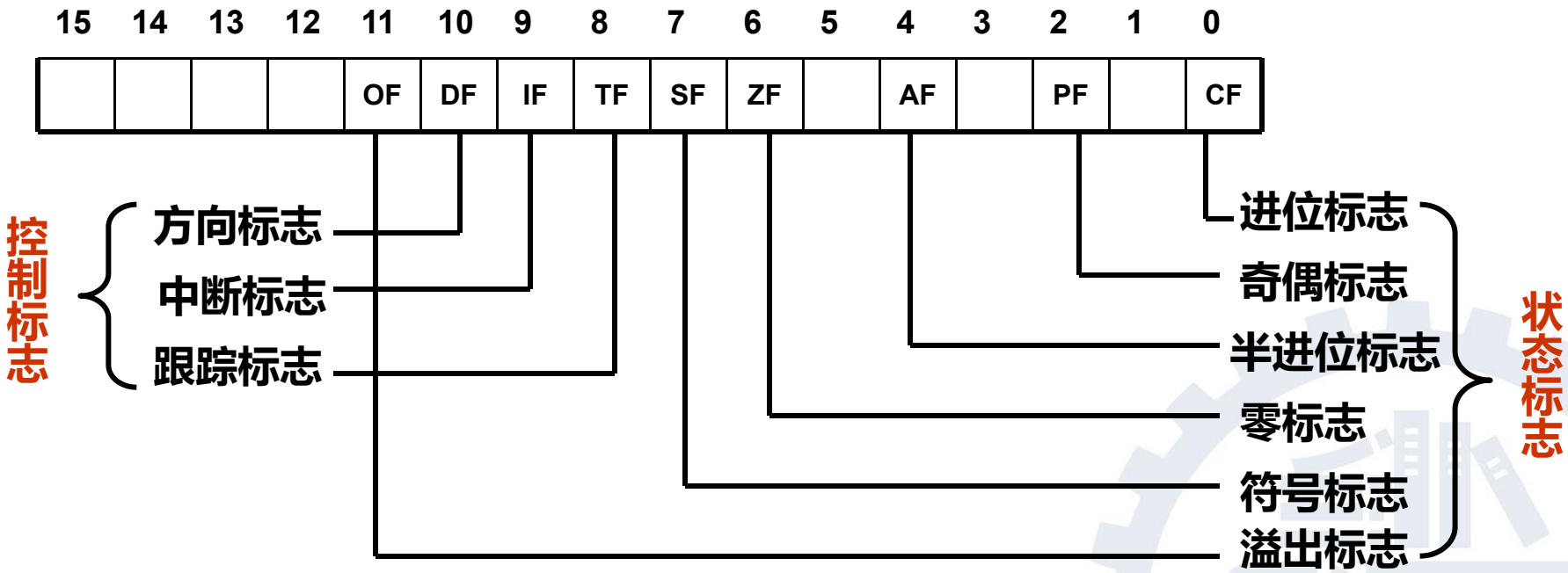
通用寄存器的专门用途

寄存器	用　　法
AX	字乘法，字除法，字I/O
AL	字节乘法，字节除法，字节I/O，十进制算术运算
AH	字节乘法，字节除法，DOS中断功能号
BX	存储器指针
CX	串操作或循环控制计数器
CL	移位计数器
DX	字乘法，字除法，间接I/O
SI	存储器指针(串操作中的源指针)
DI	存储器指针(串操作中的目的指针)
BP	存储器指针(存储堆栈指针)
SP	堆栈指针

标志寄存器（FLAGS/PSW）

- FLAGS是一个16位的寄存器，共9个标志，其中6个状态标志，3个控制标志
  - 状态标志反映EU执行算术逻辑运算后的结果特征
  - 控制标志用来控制CPU的工作方式或工作状态

标志寄存器的格式及各位的含义：



1. 状态标志——记录程序运行结果的状态信息

- 最基本的标志，有6个
- 用来记录指令执行结果的辅助信息
- 主要由加减运算和逻辑运算指令设置
- 处理器主要使用其中5个构成各种条件，分支指令判断这些条件实现程序分支

11	10	9	8	7	6	5	4	3	2	1	0
OF	DF	IF	TF	SF	ZF	0	AF	0	PF	1	CF

进位标志CF（Carry Flag）

- 当加减运算的最高有效位有进位（加法）或借位（减法）时，CF = 1，否则CF = 0

例1：00111010 + 01111100 = 10110110

最高位无进位， CF = 0

例2：10101010 + 01111100 = [1]00100110

最高位有进位，CF=1

11	10	9	8	7	6	5	4	3	2	1	0
OF	DF	IF	TF	SF	ZF	0	AF	0	PF	1	CF

- 针对**无符号整数**，判断加减结果是否超出表达范围
  - N个二进制位表达无符号整数的范围：

$$0 \sim 2^N-1$$

  - 8位：0～255
  - 16位：0～65535
  - 32位：0～2^{32} - 1

AF(Auxiliary carry Flag)辅助进位标志，当运算结果的低4位向高4位有进位或借位时，AF=1。该标志与操作数长度无关。

11	10	9	8	7	6	5	4	3	2	1	0
OF	DF	IF	TF	SF	ZF	0	AF	0	PF	1	CF

溢出标志OF（Overflow Flag）

运算时，最高位和次高位只有一个有进位，就溢出了

- **有符号数加减结果有溢出则OF = 1，否则OF = 0**

例1：00111010 + 01111100 = 10110110

超出表达范围，OF = 1

例2：10101010 + 01111100 = [1]00100110

未超出表达范围，OF=0

11	10	9	8	7	6	5	4	3	2	1	0
OF	DF	IF	TF	SF	ZF	0	AF	0	PF	1	CF

- 针对有符号整数，判断加减结果是否超出表达范围
  - N个二进制位表达有符号整数的范围：

$$-2^{N-1} \sim 2^{N-1}-1$$

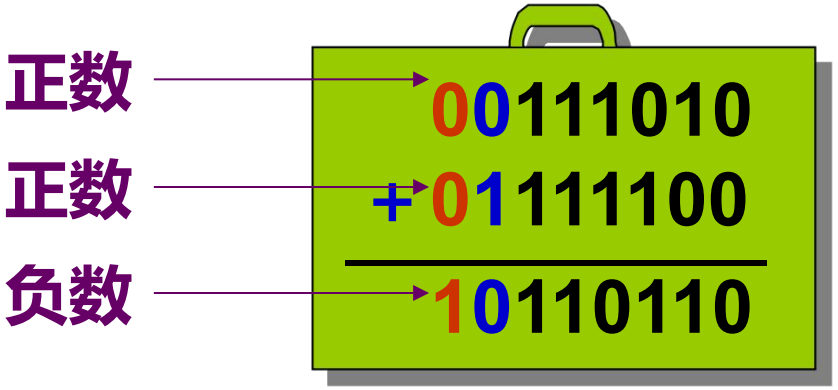
  - 8位：－128～＋127
  - 16位：－32768～＋32767
  - 32位：$-2^{31} \sim +2^{31}-1$

进位和溢出的区别

- CF反映无符号整数运算结果是否超出范围
  - 有进位，加上进位或借位后运算结果仍然正确
- OF反映有符号整数运算结果是否超出范围
  - 有溢出，运算结果已经不正确
- 处理器按照无符号整数求得结果
  - 设置进位标志CF
  - 设置溢出标志OF
- 程序员决定
  - 操作数是无符号数，关心进位
  - 操作数是有符号数，注意溢出

溢出标志的判断

- **处理器硬件判断规则**
  - 最高位和次高位同时有进位或同时无进位，无溢出；
  - 最高位和次高位进位状态不同，有溢出
- **人工判断的简单规则**
  - 只有当两个相同符号数相加（含两个不同符号数相减），而运算结果的符号与原数据符号相反时，产生溢出；其他情况下，不会产生溢出



零标志ZF（Zero Flag）

- 运算结果为0则ZF = 1，否则ZF = 0

例1：00111010 + 01111100 = 10110110

结果不是0，ZF = 0

例2：10000100 + 01111100 = [1]00000000

结果是0，ZF=1

11	10	9	8	7	6	5	4	3	2	1	0
OF	DF	IF	TF	SF	ZF	0	AF	0	PF	1	CF

符号标志SF（Sign Flag）

- 运算结果最高位为1则SF = 1，否则SF = 0

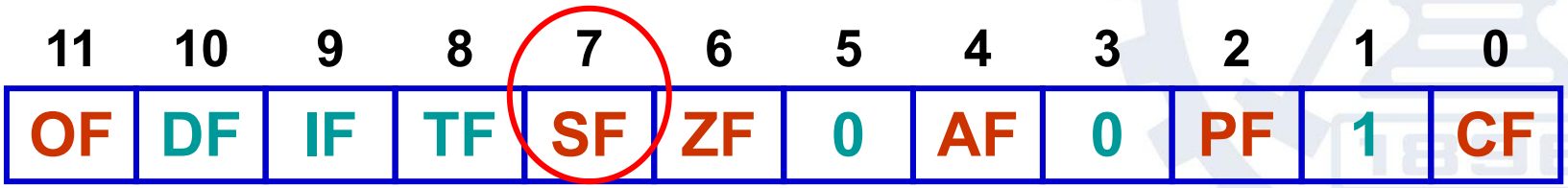
例1：00111010 + 01111100 = 10110110

最高位为1，SF = 1

例2：10000100 + 01111100 = [1]00000000

最高位为0，SF=0

进位



奇偶标志PF（Parity Flag）

- 当运算结果最低字节中”1”的个数为零或偶数时，PF = 1；否则PF = 0

例1：00111010 + 01111100 = 10110110

“1”的个数为5个，PF = 0

例2：10000100 + 01111100 = [1]00000000

“1”的个数为0个，PF=1

11	10	9	8	7	6	5	4	3	2	1	0
OF	DF	IF	TF	SF	ZF	0	AF	0	PF	1	CF

例

二进制加法

10010001B + 01110100B
= 00000101B

```
  1 0 0 1 0 0 0 1
+ 0 1 1 1 0 1 0 0
-----------------
1←0 0 0 0 0 1 0 1
进位
```

CF=1，产生进位　　AF=0，半字节无进位
SF=0，结果为正(最高位为0)　　ZF=0，结果不为0
PF=1，结果偶数个1
OF=0，将两个操作数作为有符号数，相加结果无溢出。
10010001B = -6FH，01110100B = +74H
(-6FH) + (+74H) = +05H
加法OF的判断：同符号数相加而和的符号与之相反则OF为1(产生溢出)，否则OF为0(不产生溢出)。

例

二进制减法

01110001B - 10001011B

= 11100110B

```
    01110001
  - 10001011
-------------
1 → 11100110
借位
```

CF=1，产生借位　　AF=1，半字节产生进位

SF=1，结果为负(最高位为1)　　ZF=0，结果不为0

PF=0，结果奇数个1

OF=1，将两个操作数作为有符号数，相减结果产生溢出。

01110001B = +71H，10001011B = -75H

(+71H) - (-75H) = -1AH

减法OF的判断：不同符号数相减而差的符号与减数相同则OF为1(产生溢出)，否则OF为0(不产生溢出)。

练习：下列十六进制数相加会如何影响标志位?

	8000H + 8000H 0000H	C000H + C000H 8000H	4008H + 4008H 8010H	0808H + C000H C808H
CF	1	1	0	0
PF	1	1	0	0
ZF	1	0	0	0
SF	0	1	1	1
OF	1	0	1	0

2. 控制标志——方向标志 DF(Direction Flag)

自己运行时可自主修改

- 仅用于串操作指令，控制地址的变化方向
  - 设置DF = 0，每次串操作后的存储器地址就自动增加，即从低地址向高地址处理数据串
  - 设置DF = 1，每次串操作后的存储器地址就自动减少，即从高地址向低地址处理数据串
- 执行CLD指令设置DF = 0
- 执行STD指令设置DF = 1

11	10	9	8	7	6	5	4	3	2	1	0
OF	DF	IF	TF	SF	ZF	0	AF	0	PF	1	CF

- **中断允许标志IF（Interrupt-enable Flag）：** 主要针对外中断中可屏蔽中断的开放或禁止。当IF=‘1’时，CPU允许响应可屏蔽中断，中断当前程序，转去执行中断处理程序。IF = ‘0’时，则不允许响应可屏蔽中断。用STI指令使IF置‘1’，用CLI使IF=‘0’。

- **追踪标志TF（Trace Flag）：** 用于单步调试程序。当TF=‘1’时，在执行完一条指令后，产生单步中断。这在DEBUG调试程序状态下，可以使指令单步运行，可逐一检查各寄存器内容，标志状态、存储器的检查或修改等等。TF=‘1’时为调试程序时所用，当程序调试成功后让TF=‘0’，CPU正常工作不产生单步中断。

11	10	9	8	7	6	5	4	3	2	1	0
OF	DF	IF	TF	SF	ZF	0	AF	0	PF	1	CF

2.1 80x86微处理器基本引脚与工作模式

了解几个概念

- ✓ **主频：** CPU的工作频率
- ✓ **时钟周期：** 对主频进行分频后的工作时钟
- ✓ **总线周期：** 完成一次总线操作所需的时间

  在8088CPU中，CPU与内存或接口间进行通信，

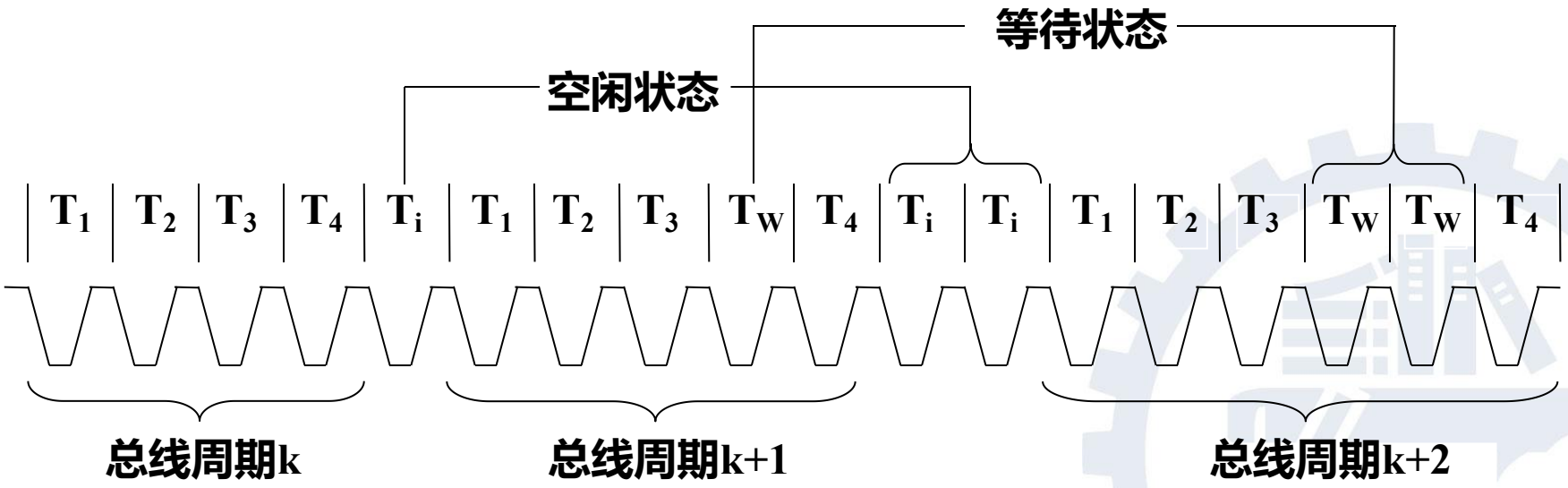
  如将一个字节写入内存一个单元(或接口)，

  或者从内存某单元(或某接口)读一个字节到CPU，

  这种读(或)写的过程称为一个总线周期。
- ✓ **指令周期:** 一条指令从开始取指令到最后执行完毕所需的时间

总线周期时序

- 1个总线周期正常情况下由4个时钟周期（T1—T4）组成；时钟周期由主频决定；
- 空闲周期T_I和等待周期T_W；
- 读、写、中断响应、总线保持与响应等的总线周期时序



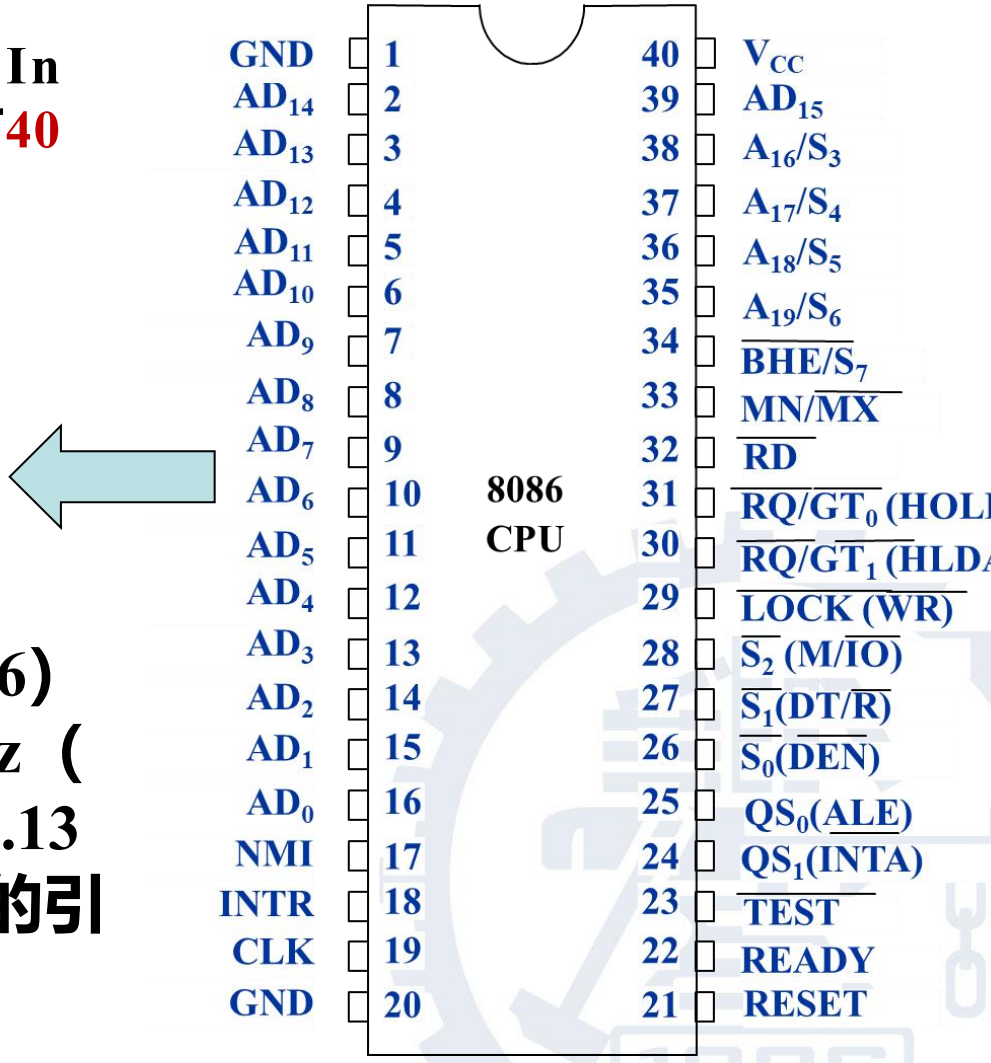
最小模式：系统中只有8086一个微处理器，所有的总线控制信号均为8086产生，系统中的总线控制逻辑电路，减少到最少。

最大模式：用于大型（中型）8086系统中。系统总是包含有两个或多个微处理器，其中一个主处理器就是8086，还有协处理器8087、8089等。通常由专门的总线控制器（8288）产生总线控制信号。



8086微处理器外部基本引脚与工作模式

- 8086采用双列直插式（Double In line Package，DIP）封装，具有40条引脚，使用+5V电源供电。



时钟频率有3种: 5MHz（8086）、8MHz（8086-1）和10MHz（8086-2）。其引脚信号如图3.13所示，括号内为最大模式时的引脚名称。

	名　称	功　能	引 脚 号	类　型
公用信号	AD_{15} ~ AD_0	地址/数据总线	2 ~ 16,39	双向、三态
	A_{19}/S_6 ~ A_{16}/S_3	地址/状态总线	35 ~ 38	输出、三态
	$\overline{BHE}/S_7$	总线高允许/状态	34	输出、三态
	$MN/\overline{MX}$	最小/最大模式控制	33	输入
	$\overline{RD}$	读控制	32	输出、三态
	$\overline{TEST}$	等待测试控制	23	输入
	READY	等待状态控制	22	输入
	RESET	系统复位	21	输入
	NMI	不可屏蔽中断请求	17	输入
	INTR	可屏蔽中断请求	18	输入
	CLK	系统时钟	19	输入
	V_{cc}	+15V 电源	40	输入
	GND	接地	1,20	
最小模式信号 ($MN/\overline{MX}=V_{CC}$)	HOLD	保持请求	31	输入
	HLDA	保持响应	30	输出
	$\overline{WR}$	写控制	29	输出、三态
	$M/\overline{IO}$	存储器输入输出控制	28	输出、三态
	$DT/\overline{R}$	数据发送/接收	27	输出、三态
	$\overline{DEN}$	数据允许	26	输出、三态
	ALE	地址锁存允许	25	输出
	$\overline{INTA}$	中断响应	24	输出
最大模式信号 ($MN/\overline{MX}=GND$)	$\overline{RQ}/\overline{GT}_{1,0}$	请求/允许总线访问控制	30,31	双向
	$\overline{LOCK}$	总线优先权锁定控制	29	输出、三态

两种模式下公用的引脚信号

- 所谓三态是指总线输出可以有三个状态：
- 高电平、低电平和高阻状态。
- 当处于高阻状态时，该总线在逻辑上与所有连接负载断开。

1. 地址总线、数据总线、状态信号

（1）AD_{15}～AD_0（2~16，39）

- 地址/数据复用总线；
- 双向、三态；
- 分时输出低16位地址信号及进行数据信号的输入/输出。
- A15-A0：T1输出访问存储器或I/O的地址信息。
- D15-D0：T2-T4输出与存储器和I/O设备交换数据信息。

(2) A19/S6 ~ A16/S3 (35~38)

- 地址 / 状态复用线;
- 输出、三态;
- 分时输出地址的高4位及状态信息。
- A19-A16：T1输出访问存储器的20位地址的高4位地址A19-A16。
- S6-S3： T2-T4输出CPU的工作状态。
  - 其中S6为0用以指示8086当前与总线连通;
  - S5为1表明8086可以响应可屏蔽中断。
  - S4、S3共有四个组合状态，用以指明当前使用的段寄存器;

S_4	S_3	当前正在使用的段寄存器
0	0	ES
0	1	SS
1	0	CS或未使用任何段寄存器
1	1	DS

(3)$\overline{\text{BHE}}$/S7（34）

- 高8位数据总线有效/状态复用引脚；
- 三态输出；
- 表示高8位数据线D_{15} ~ D_8上的数据有效和S_7 状态信号；
- T2-T4状态输出S7状态信号。 8086中无定义。

偶地址单元数据通过数据总线低8位传输。
奇地址单元数据通过数据总线高8位传输。

$\overline{\text{BHE}}$	A0	总线使用情况
0	0	从偶地址单元开始，在16位数据总线上进行字传送
0	1	从奇地址单元开始，在高8位数据总线上进行字节传送
1	0	从偶地址单元开始，在低8位数据总线上进行字节传送
1	1	无效

2. 控制总线

(1) $\overline{RD}$ (32)

- 读信号;
- 三态输出;
- 低电平有效;
- RD = 0，表示CPU正在读存储器或I/O端口。

(2) READY(22)

- 准备就绪信号;
- 输入;高电平有效。
- READY=1，表示CPU访问的存储器或IO端口已准备好传送数据。若CPU在总线周期T3状态检测到READY=0，表示未准备好，CPU自动插入一个或多个等待状态TW，直到READY=1为止。

(3) $\overline{\text{TEST}}$ (23)

- 测试信号，输入。
- 当CPU执行WAIT指令时，CPU每隔5个T状态进行一次测试；
- 当测试到TEST =1，则CPU 重复执行WAIT指令，即CPU处于空闲等待状态，
- 直到测试到TEST =0时，等待状态结束，CPU继续执行后续指令。

(4) RESET (21)

- 复位信号，输入，高电平有效。
- RESET信号至少要保持4个时钟周期。
- 复位时：标志寄存器、IP、DS、SS、ES为0，CS=FFFFH，复位后CPU从FFFF0H处开始执行。

(4) INTR (18)

- 可屏蔽中断请求信号；
- 输入，高电平有效。
- INTR=1，表示外设向CPU发出中断请求，CPU在当前指令周期的最后一个T状态去采样该信号，若此时，IF=1，CPU响应中断，执行中断服务程序。

(5) NMI (17)

- 不可屏蔽中断请求信号，
- 输入，上升沿触发。
- 该请求信号不受IF状态的影响，也不能用软件屏蔽，一旦该信号有效，则执行完当前指令后立即响应中断。

(6) $\overline{MN}/MX$ (33)

- 工作模式选择信号；
- 输入。
  - $\overline{MN}/MX$=1，表示CPU工作在最小模式系统；
  - $\overline{MN}/MX$=0，表示CPU工作在最大模式系统。



3. 其他信号

(1) VCC（40）GND（1，20）

- 电源、接地引脚
- 输入；
- 8086采用单一的+5V电源；
- 但有两个接地引脚。

(2) CLK（19）

- 时钟信号输入引脚。
- 要求时钟信号的占空比为33%；
- 即1/3周期为高电平，2/3周期为低电平。

最小模式

最小模式——仅支持单处理器

主要需解决：

地址与数据的分离
地址锁存

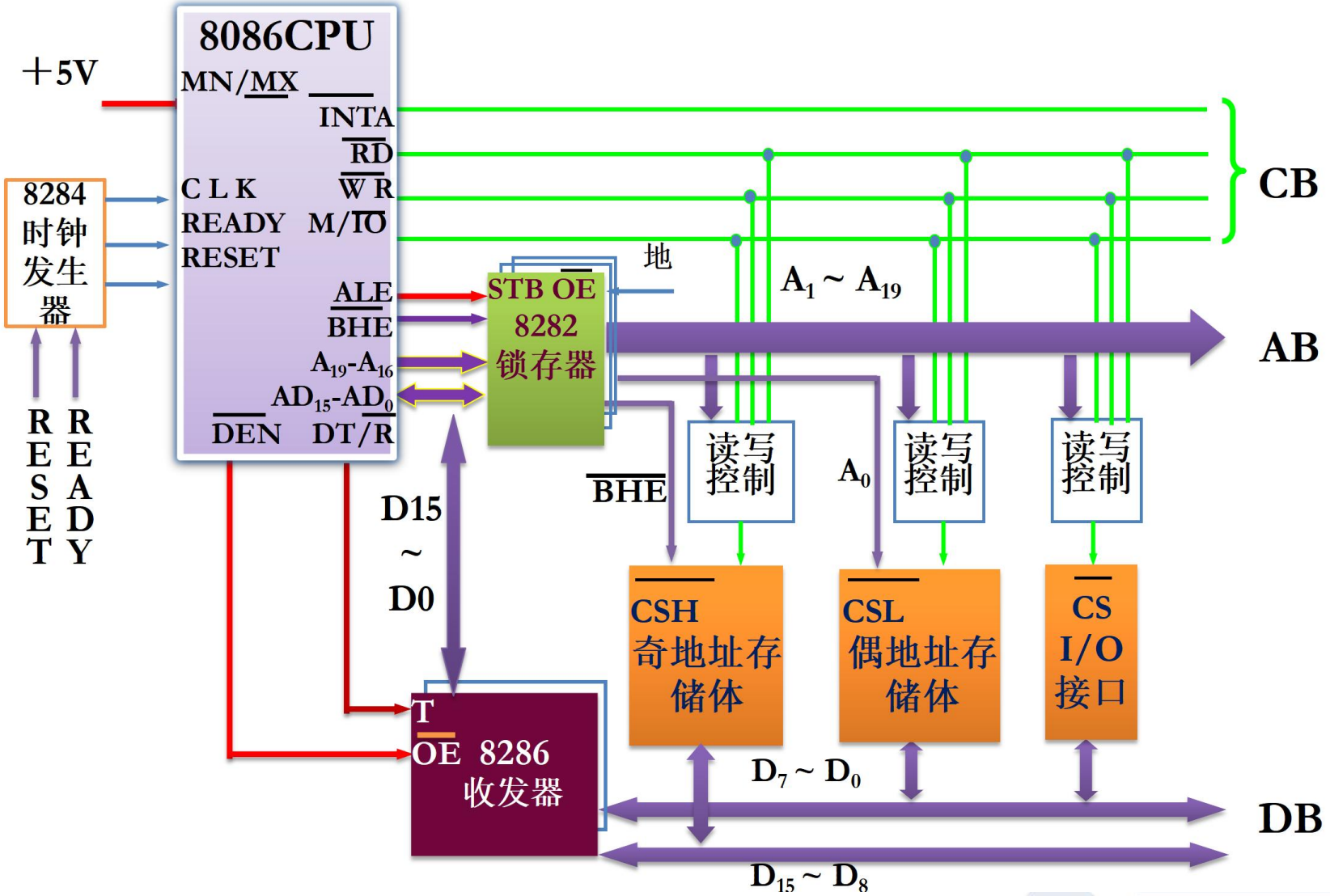
电路实现方案

用3片8位的锁存器8282实现地址锁存。ALE为锁存控制信号，OE#≡0使锁存的地址直接输出；

用2片双向三态门8286用作数据总线驱动和隔离，DT//R作为方向控制，/DEN作为开门信号；

其他控制信号由8086直接产生。

8086最小工作模式下控制核心单元的组成



（1）时钟发生器

8086内没有时钟发生电路，8284就是供8086系列使用的单片时钟发生器。它由**时钟电路、复位电路、准备就绪电路**3部分组成。

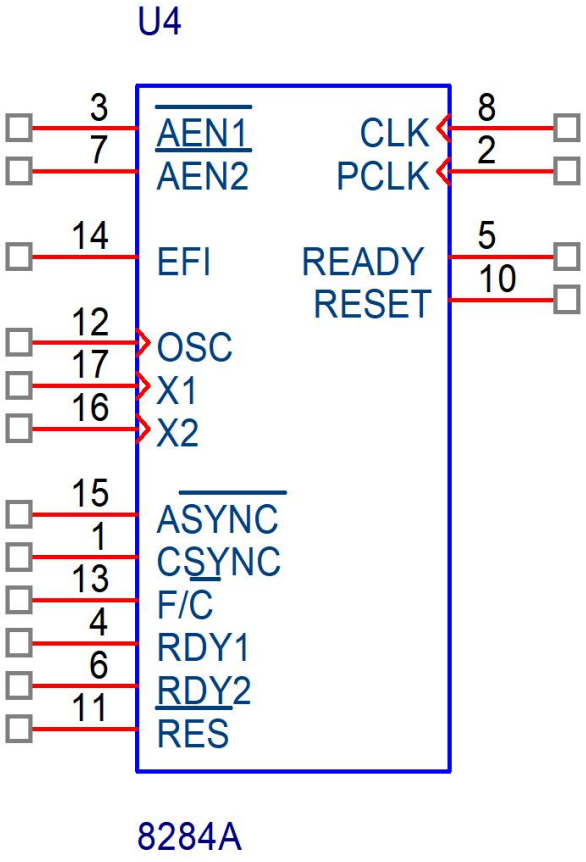
❑ **时钟发生电路**

X1,X2：外接石英晶体连接端

F/$\overline{C}$：使用外振源还是由X1, X2端外接晶体振荡器，低电平时外接晶体振荡器。

CLK：时钟信号输出端，为振荡信号经3分频后产生。

PLCK：对CLK时钟信号二分频产生，占空比为1/2，供定时/计数器使用。

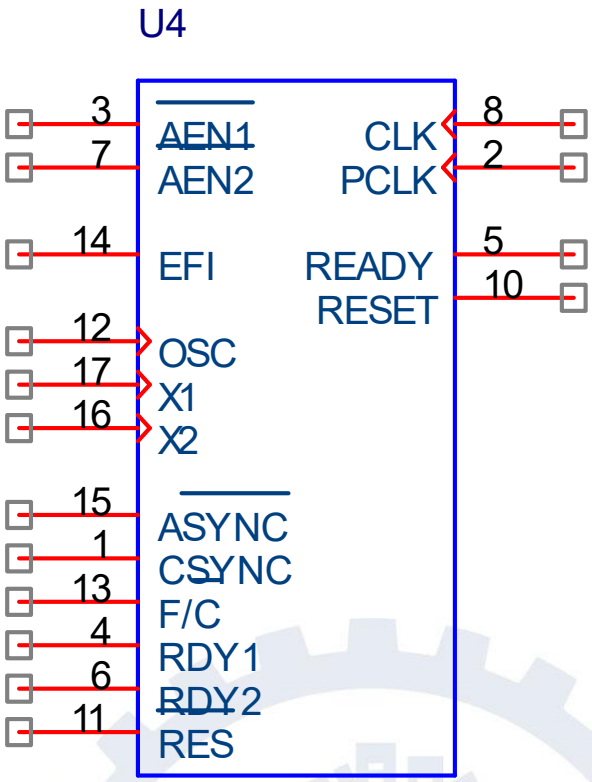


◘ 时钟发生电路

CSYNC：时钟同步输入，为多个8284同步工作而设置

EFI：外振源输入端

OSC：晶振频率输出端，供显示器用



❑复位电路

RES:输入信号，用于产生使系统复位的输出信号RESET，一般来自电源电路。

RESET:输出信号，到CPU的RESET端

❑准备就绪电路

RDY1,RDY2：准备就绪输入信号，

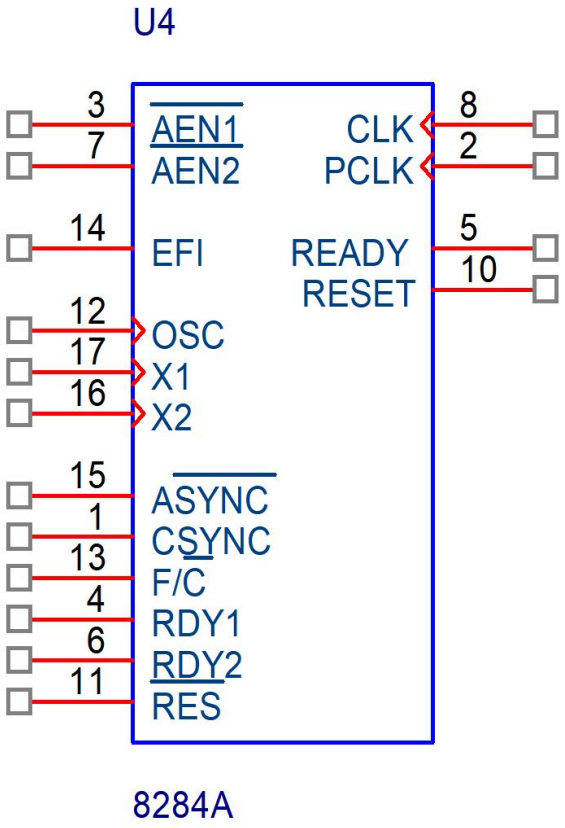
AEN1,AEN2：用来决定对应的RDY信号生效与否，若有效使RDY1和RDY2产生REDAY信号，否则使CPU产生等待周期

ASYNC：准备就绪输入信号和时钟信号进行同步的方式选择输入端

REDAY：输出到CPU的准备就绪信号

8284A的功能：

- 产生恒定的时钟信号，对准备好信号和复位信号进行同步；
- 外界控制信号RDY和RES可以在任何时候到来，8284A把它们同步在时钟下降沿时输出READY和RESET信号到CPU。

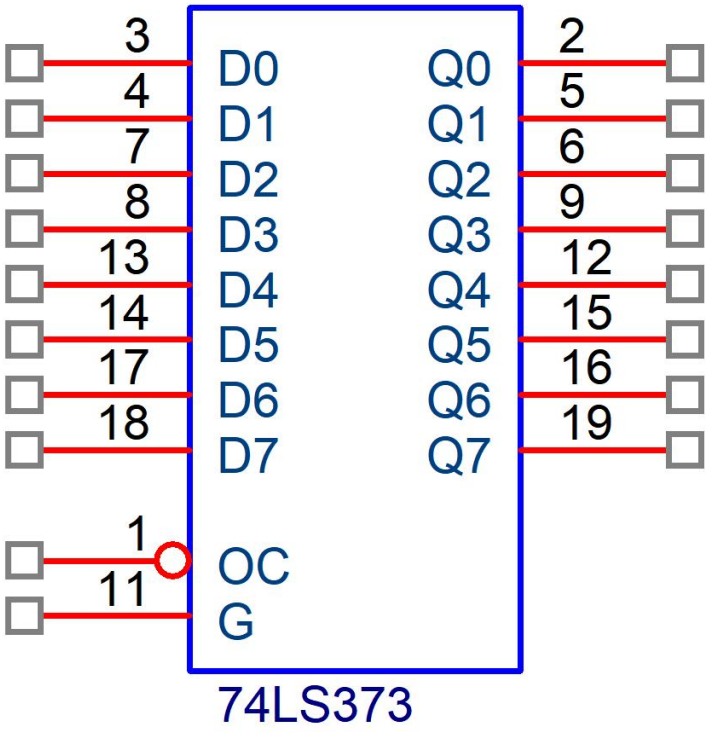


(2) 地址锁存器：

Intel8282或74LS373

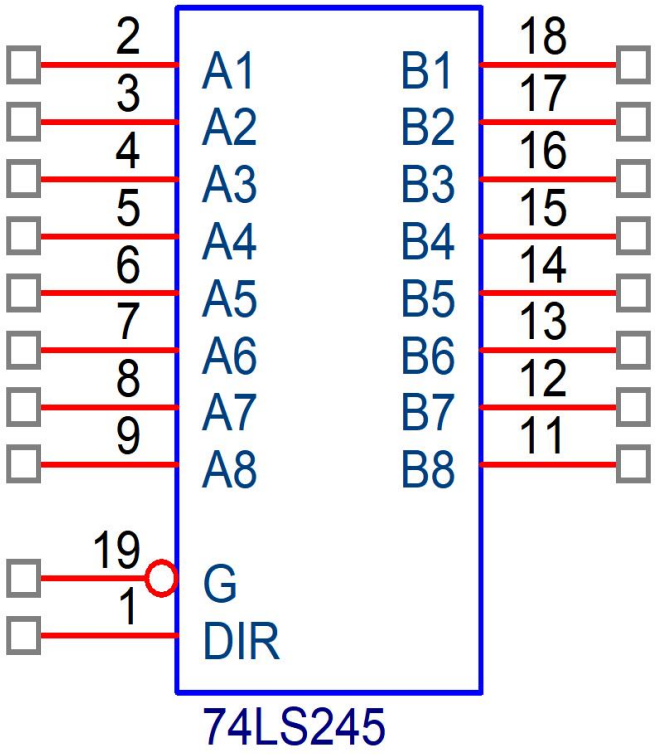
当地址锁存允许信号ALE被送到373的选通端G上时，373就锁存送到它的数据输入端的数据。

当把一个低电平有效的信号送给输出允许端OC(OE)时，373就把锁存的数据从数据输出端输出。



(3) 双向总线驱动器

8086收发数据的负载能力是有限的。为了增加8086的负载能力，尤其是组建较大系统时，在8086和系统数据总线间需使用双向总线驱动器。用于双向总线驱动器的芯片有8286和74LS245。



G：控制驱动器A端和B端何时接通

DIR：当DIR输入高电平时。数据从A传到B；当DIR输入低电平时。数据从B传到A。

.最小模式下的引脚信号

（1）$\overline{\text{INTA}}$ （24）

- 中断响应；
- 输出；
- 当CPU响应INTR时，$\overline{\text{INTA}}$=0，表示响应中断。

（2）ALE（25）

- 地址锁存允许信号；
- 输出；
- 高电平有效。
- 在任一总线周期的T1期间输出一个正脉冲，用于输出地址锁存信号。

(5) $M/\overline{IO}$ (28)

- 存储器或IO端口访问信号；三态、输出；
- $M/\overline{IO}$=1时，表示CPU当前正在访问存储器；
- $M/\overline{IO}$=0时，表示CPU当前正在访问I/O端口。

(6) $\overline{WR}$ (29)

- 写信号；
- 输出、三态；
- 低电平有效。
- WR=0，表示CPU当前正在写存储器或 I/O 端口。

- 最小模式下，信号$M/\overline{IO}$，$\overline{RD}$，和$\overline{WR}$组合起来决定了系统中数据传输的方式。

总线周期	M/IO	DT/R	WR
存储器读	1	0	1
存储器写	1	1	0
I/O读	0	0	1
I/O写	0	1	0

(7) HOLD (31)

- 总线请求；
- 输入；
- 用于其它主控器（处理器、DMA等）向本CPU 请求占用总线。

(8) HLDA (30)

- 总线请求响应，输出；
- 高电平有效；
- 表示CPU认可其他总线部件提出的总线占用请求，准备让出总线控制权。

最大模式

- 在最小模式下，CPU必须配有8282、8284、8286才能构成整个微处理器系统，才能通过地址、数据、控制总线与外部设备联系，控制外设工作。
- 在最大模式下，除了上述还需要配置8288总线控制器，由8288对处理器发出的控制信号进行变换和组合，最终由8288产生总线控制信号，而不是由CPU直接产生。
- 有两个或两个以上的CPU，一个为主处理器8086/8088， 另一个为协处理器8087/8089。数值运算协处理器8087, 输入输出协处理器8089。

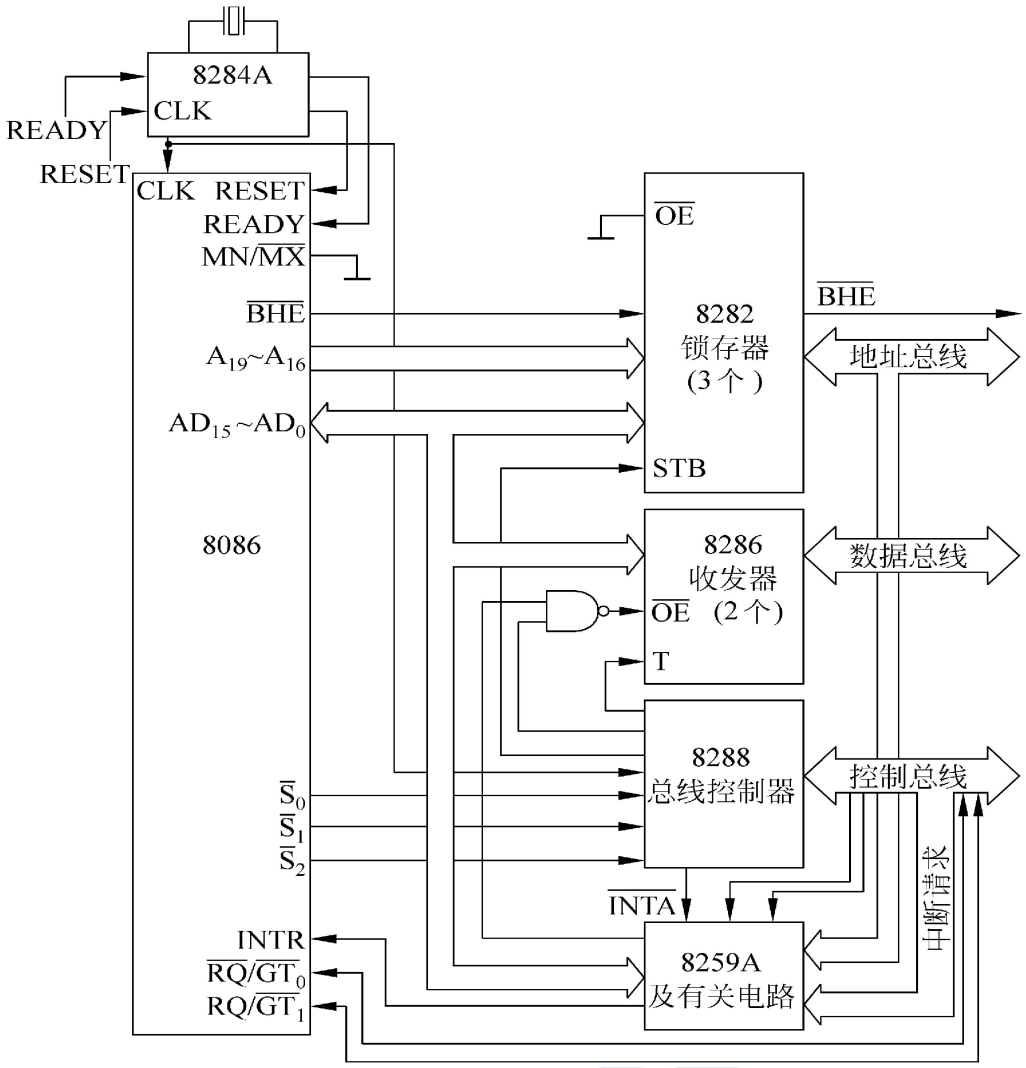
1. 8086在最大模式下的典型配置

与最小模式系统相比较，主要区别是最大模式系统中增设了**一个总线控制器8288和一个总线仲裁器8289。**

8086CPU输出的状态信号S2～S0同时送给8288和8289，**由8288输出8086CPU系统所需要的控制信号，以得到对存储器和I/O端口的读写信号和对锁存器8282及对总线收发器8286的控制信号。**

而8289总线仲裁器对系统中多个处理器提出共享总线资源的要求作出裁决。因此，8086的最大模式系统由于8288和8289的存在，可以构成一个多处理器系统。

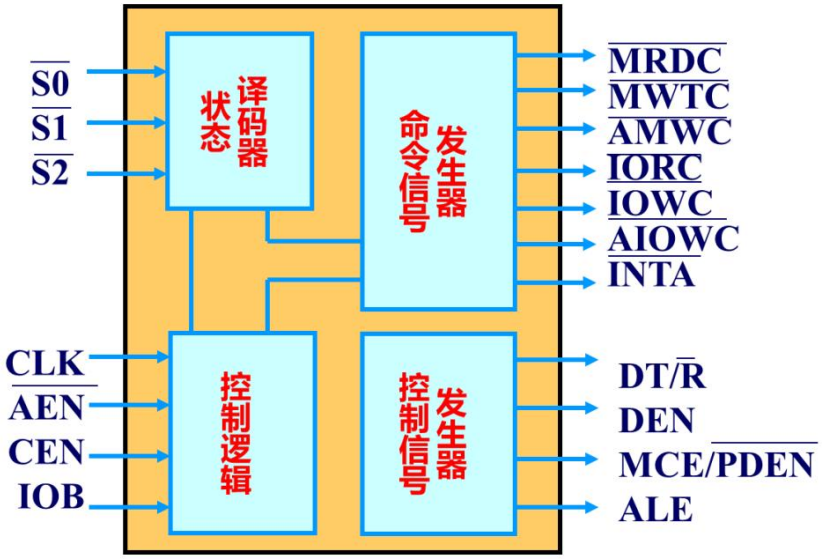
- **8086在最模式下的典型配置**



(1) 8288总线控制器

当8088工作在最大组态方式时，就需要使用8288总线控制器来产生存储器和I/O端口读写操作的控制信号。

在最大组态的系统中，命令信号和总线控制所需要的信号都是8288根据8088提供的状态信号S0、S1、S2输出的。



① 不同状态下8288对应的输出命令信号

$\overline{S_2}$	$\overline{S_1}$	$\overline{S_0}$			
0	0	0	中断响应	$\overline{INTA}$	
0	0	1	读I/O	$\overline{IORC}$	
0	1	0	写I/O	$\overline{IOWC}$	$\overline{AIOWC}$
0	1	1	暂停		
1	0	0	取指	$\overline{MRDC}$	
1	0	1	读存储器	$\overline{MRDC}$	
1	1	0	写存储器	$\overline{MWTC}$	$\overline{AMWC}$
1	1	1	无效		

8086微处理器外部基本引脚与工作模式

8288各输出命令信号的功能

$\overline{INTA}$ 用于输出中断响应

$\overline{IORC}$ 读I/O信号

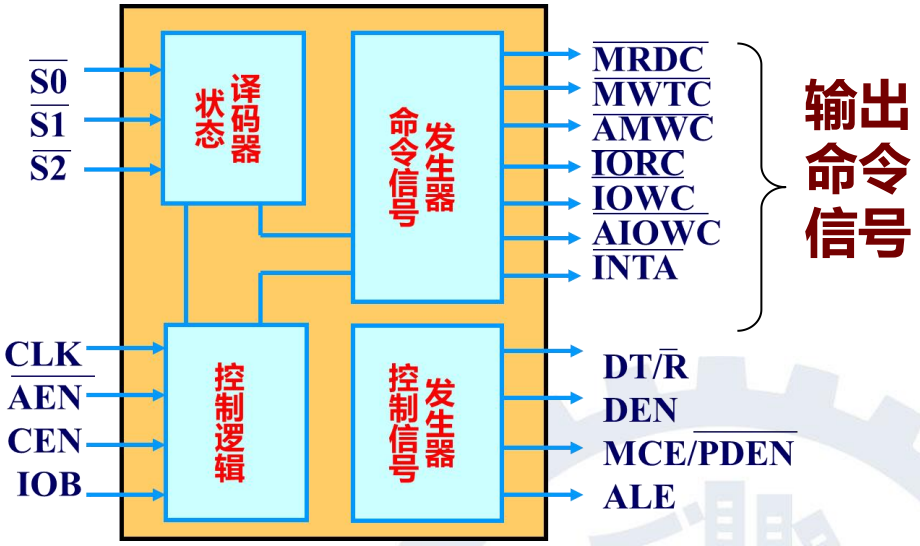
$\overline{IOWC}$ 写I/O信号

$\overline{AIOWC}$ I/O写超前控制信号

$\overline{MRDC}$ 读存储器信号

$\overline{MWT}$ 写存储器信号

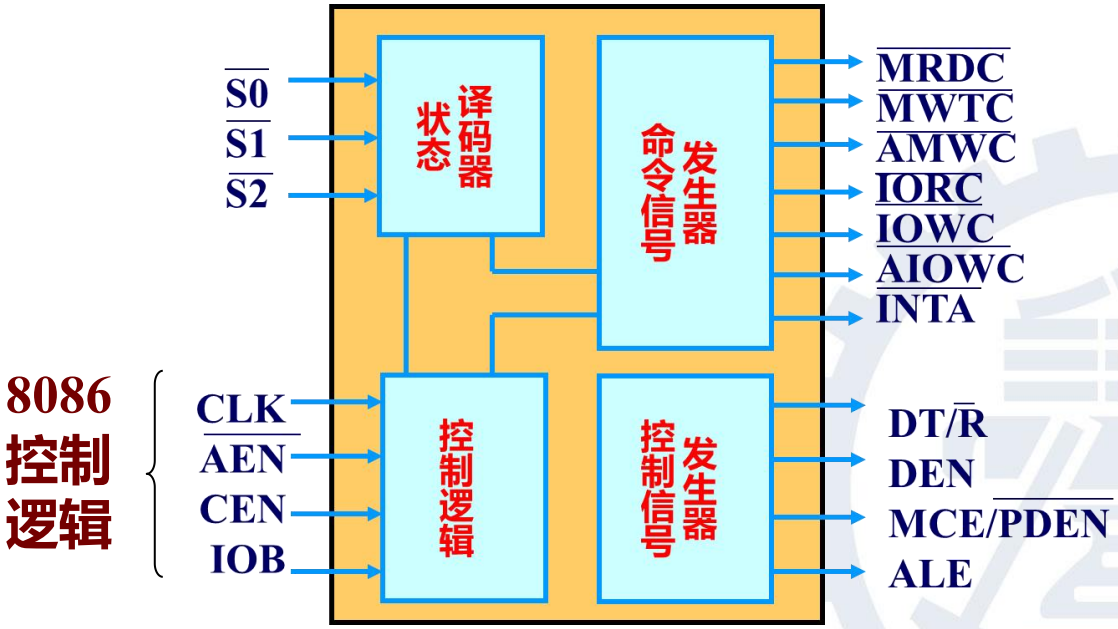
$\overline{AMWC}$ 存储器写超前控制信号



8086微处理器外部基本引脚与工作模式

② 8288控制逻辑

8288的工作受输入控制信号控制，控制信号包括IOB、$\overline{AEN}$、CEN和CLK，其功能是使8288与CPU保持同步工作。



8086微处理器外部基本引脚与工作模式

IOB：总线工作方式信号，决定该8288的工作方式。

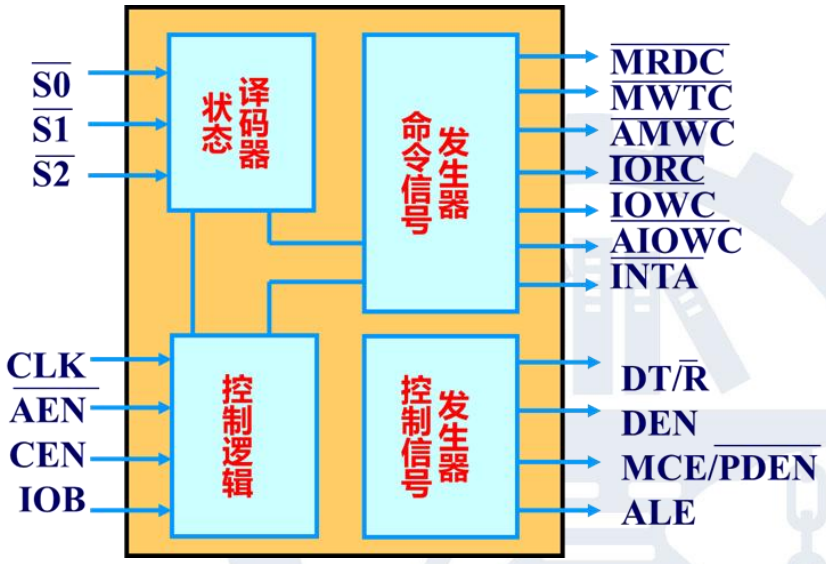
IOB { =0，系统总线工作方式
IOB { =1，I/O总线工作方式

$\overline{AEN}$：地址使能信号，当8288处于系统总线方式下（IOB=0），用于多总线同步。该信号保持低电平115ns后，8288开始输出控制信号。

CEN：8288片选有效信号，用于多片8288协调工作。

CEN { =1，该8288工作
CEN { =0，该8288不工作

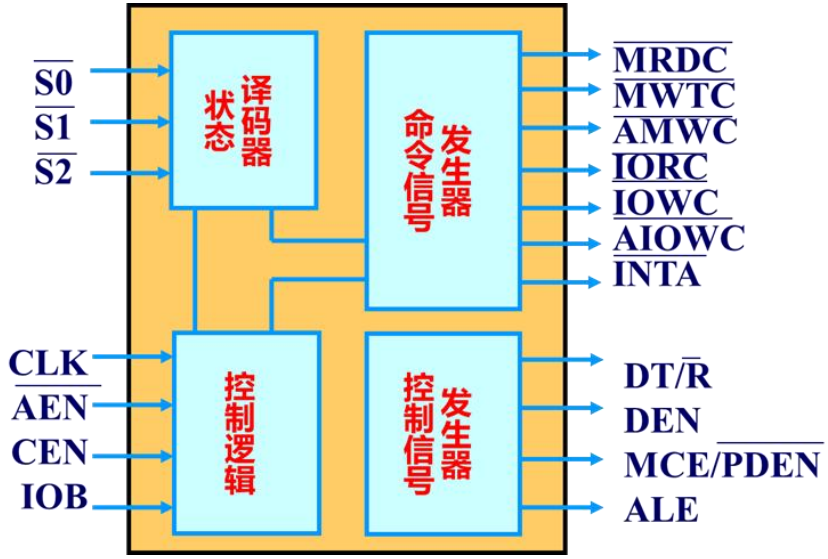
CLK：时钟信号，通常由系统时钟提供，同步控制信号的节拍。



3.6 8086微处理器外部基本引脚与工作模式

③ 8288输出控制信号

8288在进行总线读写控制中，需要提供相应的地址锁存、数据使能及方向控制等信号。8288提供的信号包括ALE、DEN、DT/$\overline{R}$和MCE/$\overline{PDEN}$。



ALE：地址锁存允许信号

DEN：数据使能信号

DT/$\overline{R}$：数据收发方向信号

MCE/$\overline{PDEN}$：主设备使能/外设数据允许，复用信号

最大模式8086CPU外部引脚

① $\overline{S_0}$,$\overline{S_1}$,$\overline{S_2}$(Bus Cycle Status)

- 总线周期状态信号,三态输出
- 它们提供当前总线周期中所进行的数据传输过程的类型。由总线控制器8288根据这些对存储器及I/O进行控制 .

② QS_1,QS_0

- 指令队列状态,向外部输出.
- 用来表示CPU中指令队列当前的状态.编码如下:

QS_1	QS_2	含　义
0	0	无操作
0	1	从队列中取第一个字节
1	0	队列已空
1	1	从队列中取后续字节

③ $\overline{RQ}/\overline{GT_1}$, $\overline{RQ}/\overline{GT_0}$ (Request/Grant)

- 请求/同意信号,双向,低电平有效.
- 输入时表示其它主控者请求使用总线
- 输出时表示CPU 对总线请求的响应信号
- 两条线可同时与两个主控者相连。但是$\overline{GT_0}$的优先级比$\overline{GT_1}$的高。

④ $\overline{LOCK}$ 封锁信号

- 三态输出,低电平有效.
- $\overline{LOCK}$有效时表示CPU不允许其它总线主控者占用总线.
- 可防止8086在响应中断时总线被其它主控部件所占用，在中断过程中，也自动变为低电平

- 3.7 8086微处理器的时序

8086的主要操作:

① 系统的复位和启动操作;

② 暂停操作;

③ 总线操作;

④ 中断操作;

⑤ 最小模式下的总线保持;

⑥ 最大模式下的总线请求/允许。

系统的复位与启动

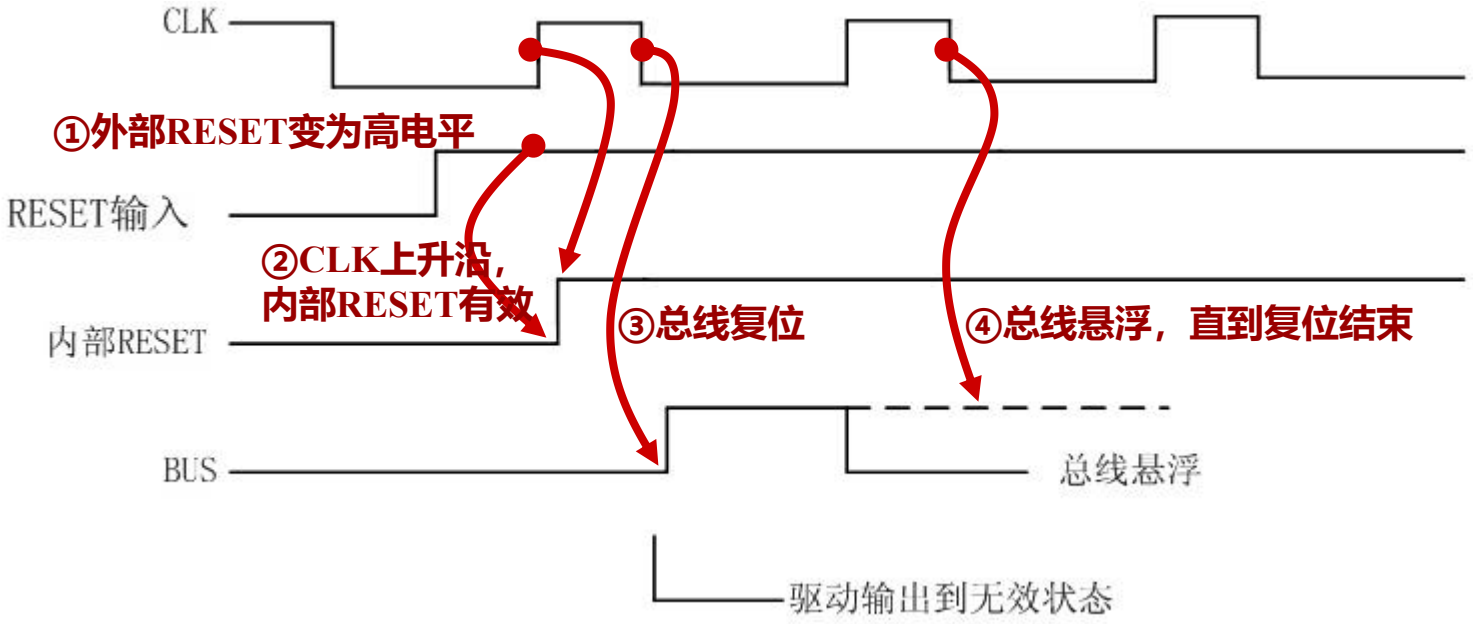
产生：RESET端上的高电平维持4个时钟周期，可使CPU复位。如果是初次加电引起的复位，则要求维持不小于50us的高电平。

CPU复位：
PSW、DS、ES、SS、IP等寄存器，指令队列被清零。
CS寄存器设置为FFFFH。
注：由于复位后，IF=0，处关中断状态，所以在初始化程序中应开中断，使CPU可响应中断请求。

CPU重启：总线周期：又称为机器周期，是CPU从存储器或 I/O设备读或写一个字节或字所需要的时间。总线周期通常包括4个时钟周期：T1、T2、T3、T4。

复位信号RESET从高电平到低电平的跳变会触发CPU内部的复位逻辑电路，当RESET由高电平变低电平7个机器周期后，CPU开始从FFFF0处执行程序。

8086微处理器的时序



复位操作时序图

复位时总线状态

- **地址线为高阻态，直到RESET变为低电平，开始从FFFF0H单元取指令；**
- **ALE、HLDA等信号为低电平（无效）；**
- **一些信号呈高阻态。**

- 3.7 8086微处理器的时序

8086的复位操作

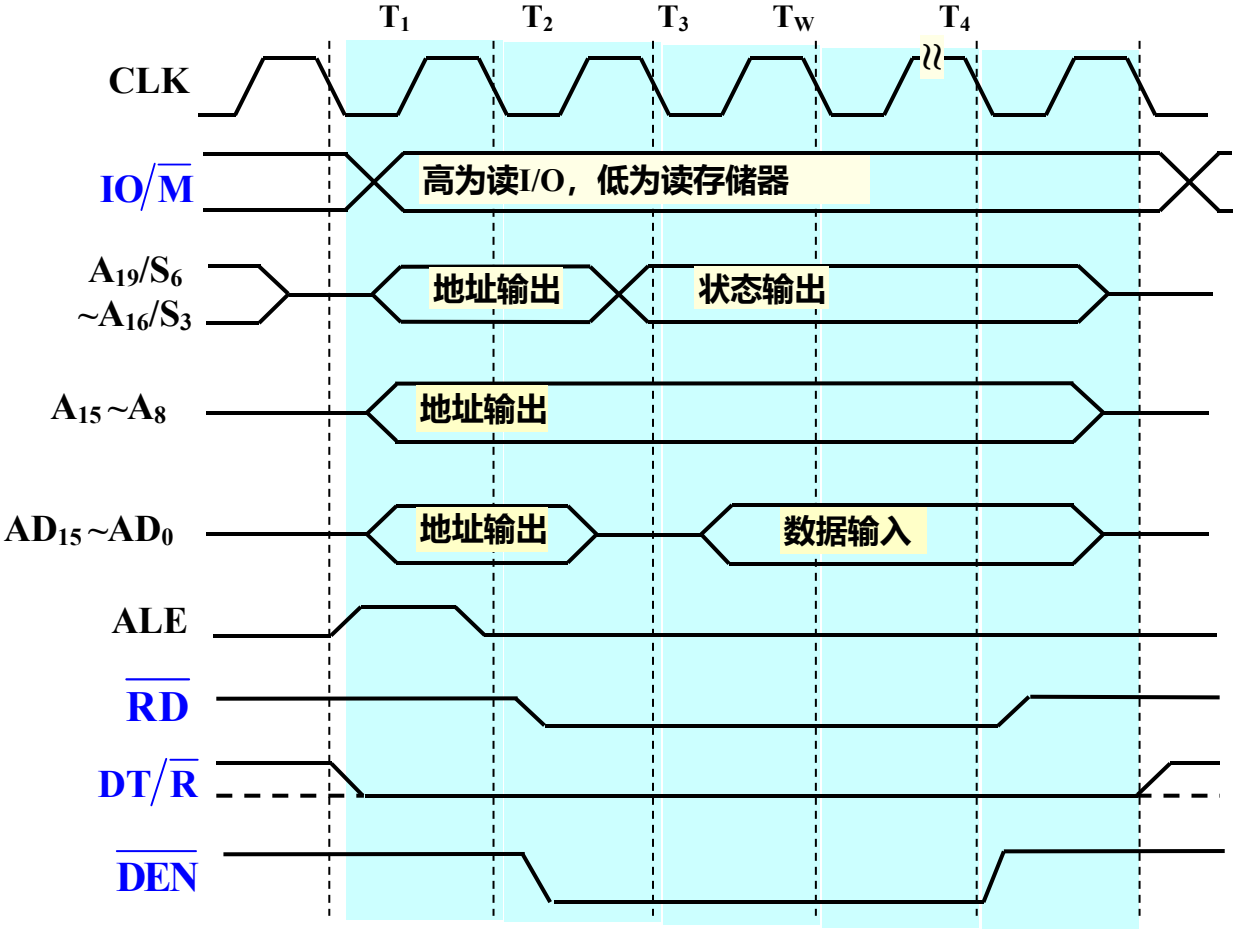
通过RESET引脚信号执行，至少维持4个时钟周期高电平，初次加电不少于50μs。

复位后各寄存器值

标志寄存器	清零
IP	0000H
CS	FFFFH
DS	0000H
SS	0000H
ES	0000H
指令队列	空
其它寄存器	0000H

最小模式系统总线周期时序

1. 8086最小模式存储器和I/O读总线周期



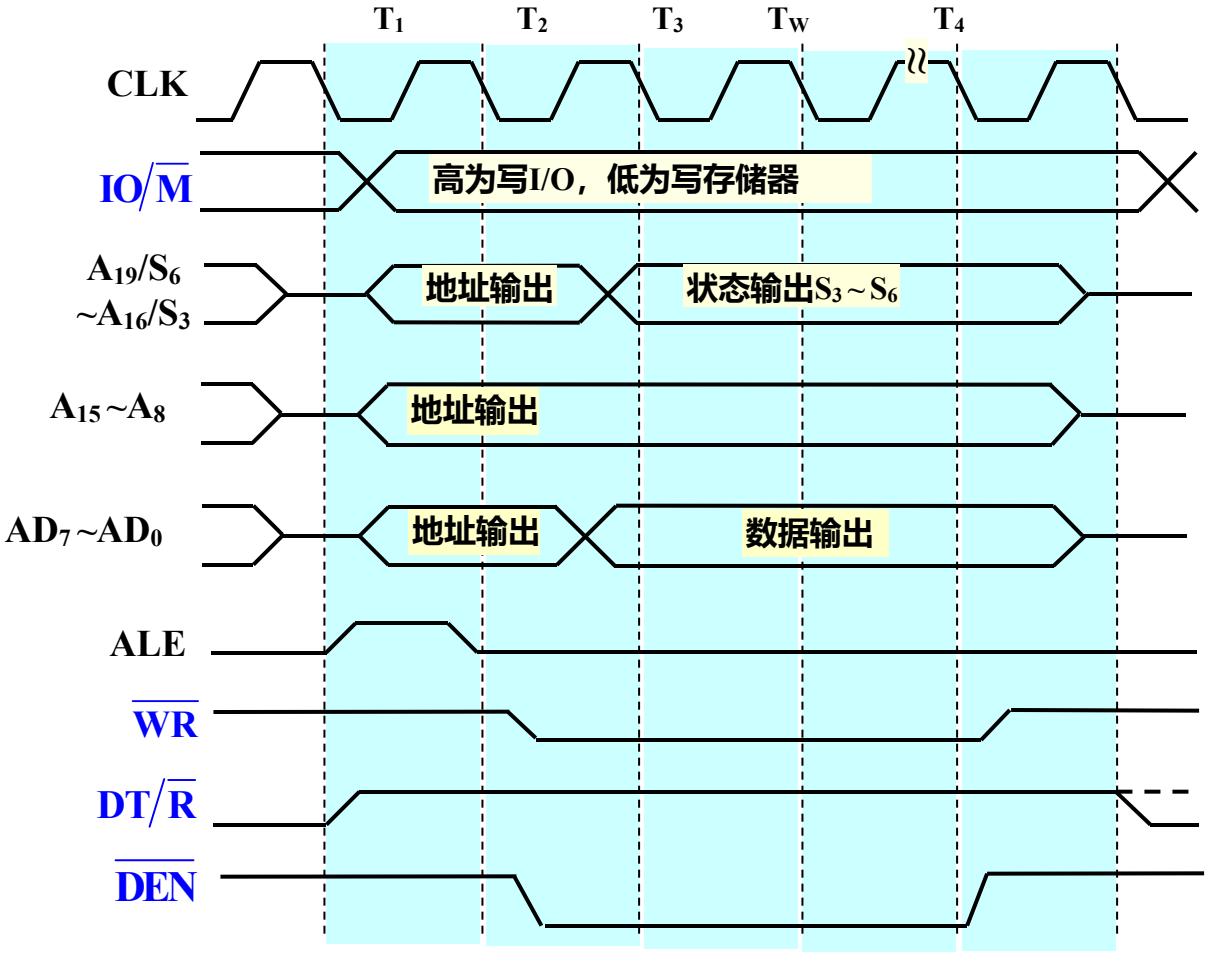
4. T_4 状态：

① T_4 状态的下降沿，CPU从AD_0~AD_7数据总线上读取数据。

② DEN，RD引脚上控制信号复位。

③ S_3~S_6状态线复位。

2. 8086最小模式存储器和I/O写总线周期



4. T_4 状态：

① T_4 状态的下降沿，完成从AD_0～AD_7数据到存储器或I/O端口总线上写数据 。

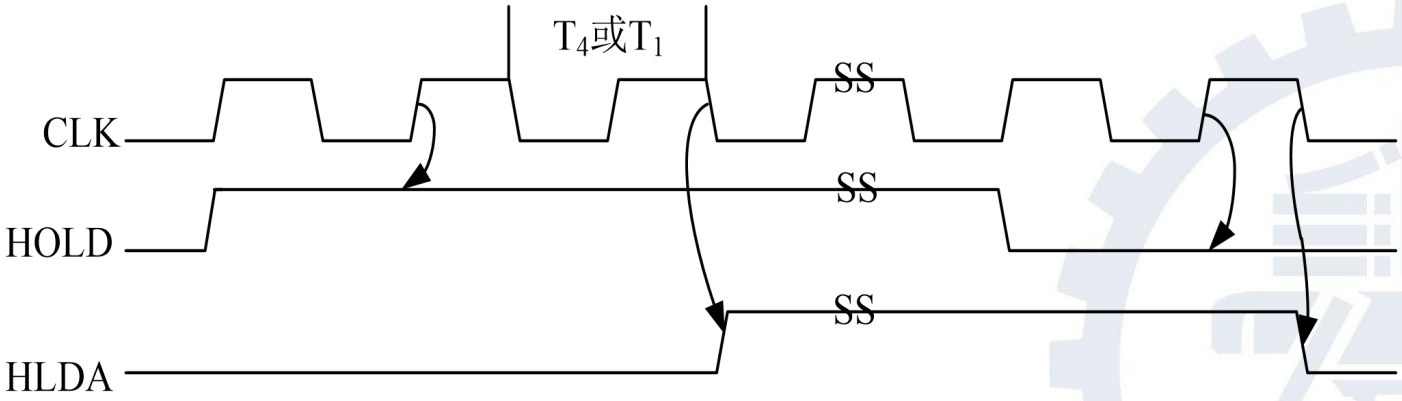
② ， 引脚上控制信号复位。

③ S_3～S_6状态线复位。

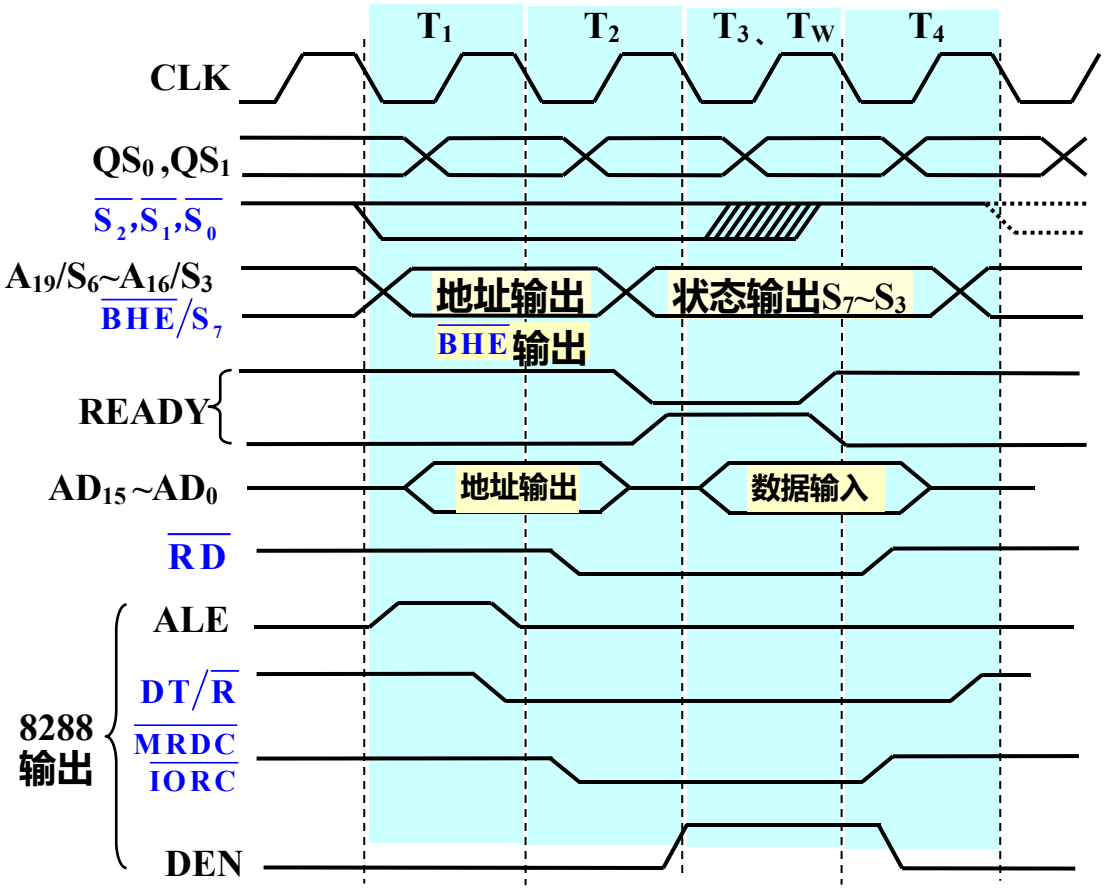
- **3.总线保持请求和总线授权时序**

当系统中CPU之外的总线主设备（例如DMA）需要占用总线时，向CPU发出一个总线保持请求信号HOLD，该信号可能与时钟信号不同步。

CPU在每个时钟周期的上升沿检测到该信号时，在当前总线周期的T4后或下一个总线周期的T1后，CPU发出HLDA信号，并让出总线。



1. 8086最大模式存储器和I/O读总线周期



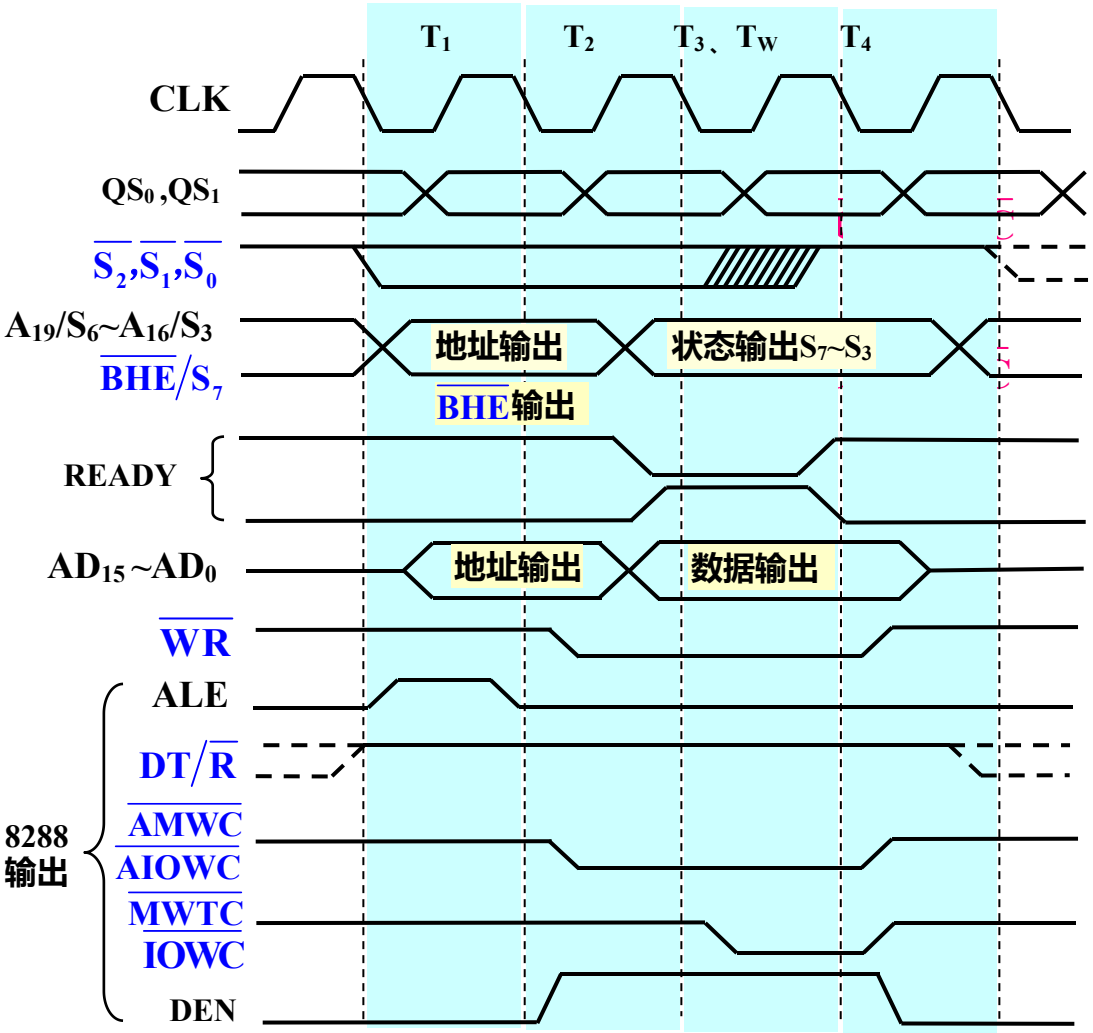
4. T_4 状态：

① T_4 状态的下降沿，CPU从AD_0 ~AD_{15}数据总线上读取数据 。

② 各引脚上控制信号复位，准备下一个总线周期。

③ S_3 ~S_6 状态线复位。

2. 8086最大模式存储器和I/O写总线周期



4. T_4 状态：

① T_4 状态的下降沿，从AD_0 ~AD_{15}数据总线到存储器或I/O端口上写数据 。

② 各引脚上控制信号复位，准备下一个总线周期。

③ S_3 ~S_6 状态线复位。

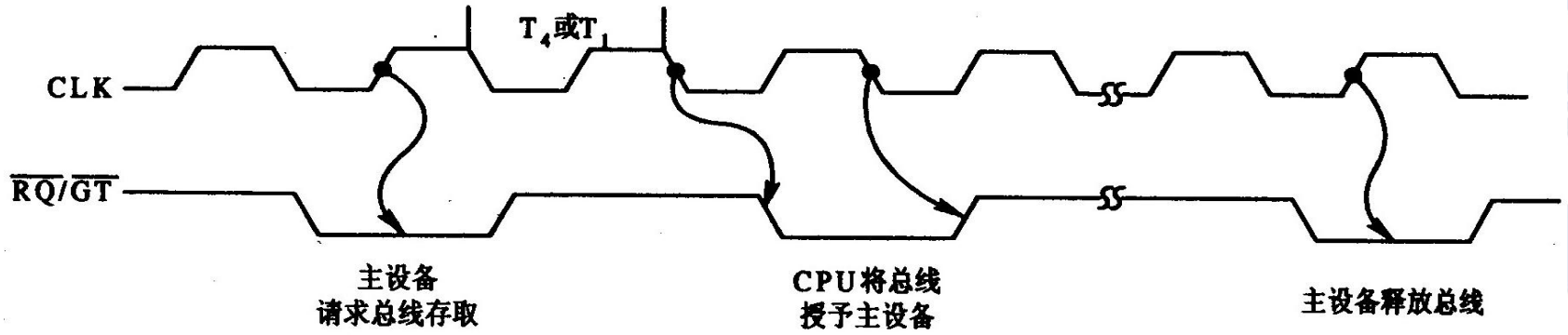
- **3.总线保持请求和总线授权时序**

以下情况下，CPU在当前总线周期开始时发出总线授予信号：

（1）CPU前一次总线传送不是对奇地址单元读或写一个字的低位字节；

（2）前一个总线周期内未出现中断响应的第1个负脉冲；

（3）不再执行带LOCK前缀的指令。



(3) DEN (26)

- 数据允许信号；三态、输出；
- DEN通常作为数据收发器的选通信号；
- 仅当DEN=0时，才允许收发器收发数据。

(4) DT/$\overline{R}$ (27)

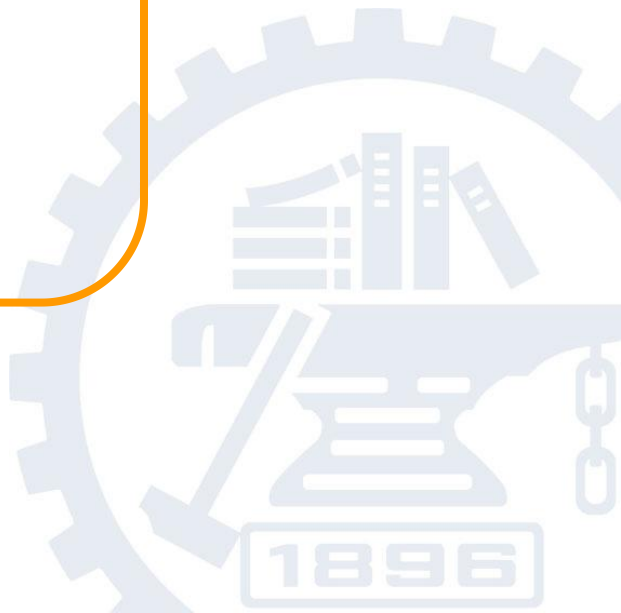
- 数据发送/接收信号；
- 三态、输出。
  - 为提高CPU数据总线驱动能力，常常使用数据收发器（8286/8287）；
  - DT/$\overline{R}$控制数据收发器的数据传送方向。
  - DT/$\overline{R}$=1时，表示CPU输出（发送）数据；
  - DT/$\overline{R}$=0时，表示CPU输入（接收）数据。

主要内容

2.1 计算机系统

2.2 8086微处理器

2.3 **存储器地址**



存储器的组织--8086实模式

- 存储器的组织采用**分段方式**，20位的**物理地址**由16位的**段地址**和16位的**偏移地址**形成，每个段的最大寻址空间为64KB； 段地址必须是16的倍数，即末尾4位必须为0。

1. 存储单元的地址和内容

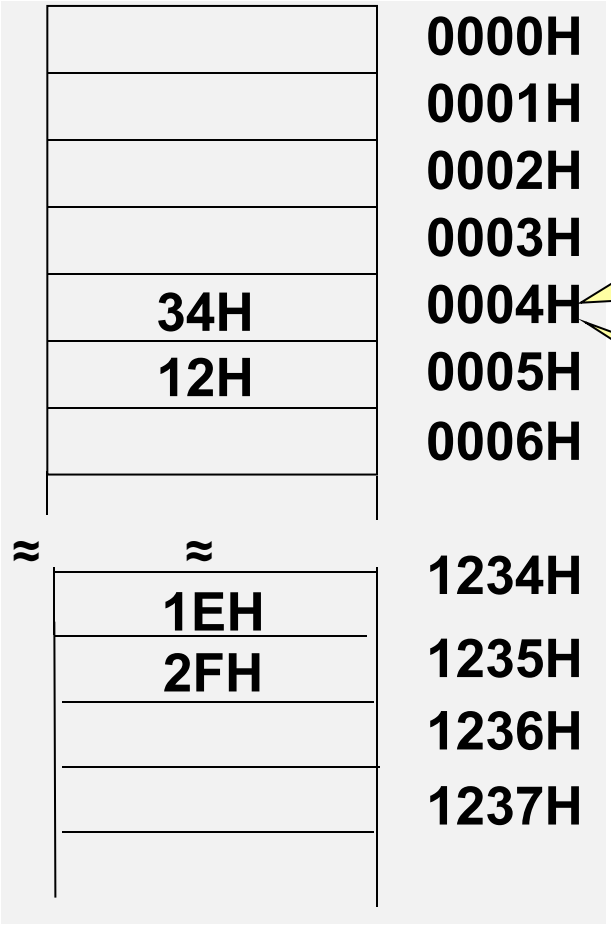
- **存储单元的地址**
  - 将存储单元编号，这个编号就是**存储器地址**
  - 用**十六进制数**来表示地址

例如：存储单元地址1234H

- **存储单元的内容**
  - 存储单元中存放的信息
  - 存储地址加括号”( )”(编程时用[])

例如：X表示某存储单元的地址，则X单元的内容可以表示为 (X)

又如：X单元中存放着Y，而Y是另一个存储单元的地址，则Y单元的内容表示为 (Y)=((X))



4号字节单元存放的信息为34H，表示：(0004H)=34H

4号字单元的内容为1234H，表示为：(0004H)=1234H

字单元由两个字节单元组成，其地址采用它的低地址来表示。

字存入存储器：低位字节存入低地址单元，高位字节存入高地址单元。

例：存储单元的地址

已知：

(0004H) = 5678H

字地址　字内容

(0004H) = 78H

字节地址　字节内容

问：

((0004H)) = ?

地址的地址　字或字节内容



存储器的分段

8086的内部寄存器是16位，地址是20位，地址宽度大于字长。显然不能用16位的寄存器一次操作实现对2^{20} = 1M字节单元的寻址。

20 根地址线地址范围 00000H ~ FFFFFH （1MB）

物理地址：每一个存储单元有一个唯一的20位地址

机器字长16位：仅能表示地址范围 0000H ~ FFFFH（64KB）

为此，引入了存储器“分段”的概念，即把1M字节内存空间分成最大64KB的段 - 可由16位寄存器进行寻址。

段地址：表示一个段的开始

偏移地址：在段内相对于段起始地址的偏移值

优点：允许程序在存储器内重定位；

有利于程序和数据的分离。

小段的首地址		小段的尾地址
00000 H	~	0000F H
00010 H	~	0001F H
00020 H	~	0002F H
…		
FFFF0 H	~	FFFFF H

逻辑地址

段地址：偏移地址

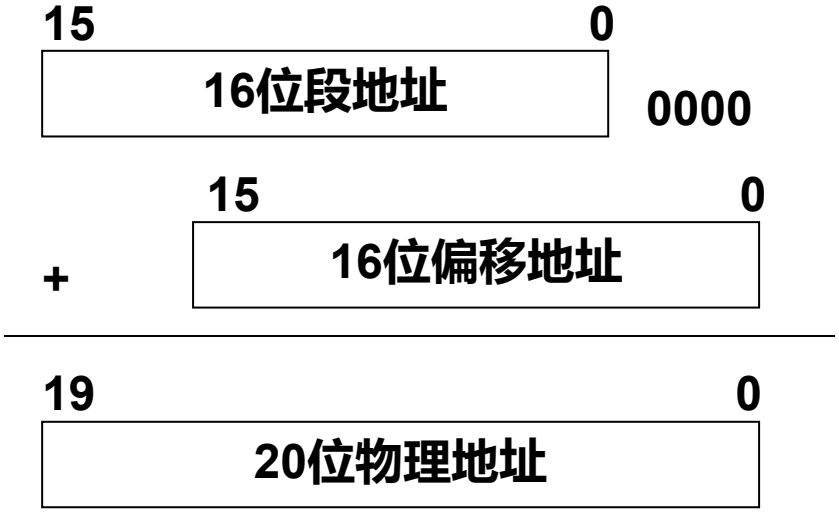
每16个字节为一小段，共有64K个小段

物理地址的形成

物理地址：每一个存储单元有一个唯一的20位地址

表示范围：00000H～FFFFFH 。

物理地址形成：段地址左移4位再加上偏移地址值



访问存储器的实际地址

这里左移的4位是二进制位；如果用十六进制表达地址就是左移一位。
左移4位还可以表达为乘以16，即：段地址×16＋偏移地址。

存储器的逻辑地址与物理地址

逻辑地址 段地址：偏移地址	内容	物理地址
1000：0000H	10011111	10000H
1000：0001H	00100110	10001H
1000：0002H	01001000	10002H
1000：0003H	10000011	10003H
1000：0004H	01011100	10004H
1000：0005H	10100010	10005H

字节内容（10000H）= 9FH;　（10001H）= 26H

字内容　（10000H）= 269FH;　（10001H）= 4826H

逻辑地址与物理地址的转换

例

段地址 = 2354H = 0010 0011 0101 0100 B

偏移地址 = 78DAH = 0111 1000 1101 1010 B

物理地址 = 2354H × 10H + 78DAH = 2AE1AH

0010 0011 0101 0100	2 3 5 4 0
+ 0111 1000 1101 1010	+ 7 8 D A
0010 1010 1110 0001 1010	2 A E 1 A

物理地址 = 段地址×16D+偏移地址 = 段地址×10H+偏移地址

逻辑地址与物理地址的转换

例

设某操作数存放在数据段，DS=250AH，数据所在单元的偏移地址=0204H。则该操作数所在单元的物理地址为：252A4H

物理地址 = 段地址×16D+偏移地址 = 段地址×10H+偏移地址

逻辑地址、物理地址

- 逻辑地址的形式：
  指令操作数，指令间的相对地址
- 物理地址：MEM的绝对地址
- 逻辑地址到物理地址的转换过程因CPU ARCH的不同而不同

C语言编程和汇编语言编程在格式上的差异

例:用C语言编程实现 c = a + b，并在屏幕上显示出结果。

```
#include "stdio.h"
int main  (int argc, char* argv[])
{
        int a=1,b=2,c;

        c=a+b;
        printf("c=%d\n",c);
        return 0;
}
```

编译后的目标文件达到3.59KB

用汇编语言实现同样的功能

```
data    segment
a       db      ?
b       db      ?
c       db      ?
string  db      'c=$'
data    ends
code    segment
main    proc far
        assume cs:code,
        ds:data, es:data
start:
        push    ds
        sub     ax,ax
        push    ax
        mov     ax,data
        mov     ds,ax
        mov     es,ax
        mov     a,1
        mov     b,2
        mov     al,a
        add     al,b
        mov     c,al
        lea     dx,string
        mov     ah,09
        int     21h
        add     c,30h
        mov     dl,c
        mov     ah,2
        int     21h
        mov     dl,0ah
        int     21h
        mov     dl,0dh
        int     21h
        ret
main    endp
code    ends
        end     start
```

汇编后的目标文件只有208字节

段地址与段寄存器

- 段是安排相关代码或数据的主存区域
- 段寄存器表明段在主存中的位置
- 4个16位段寄存器：CS DS SS ES
  - 代码段CS主要存放运行的程序。
  - 数据段DS存放运行程序所用的数据。
  - 堆栈段SS定义了堆栈的所在区域。
  - 附加段ES是附加的数据段，它是一个辅助的数据区，在串操作指令中用到目的串必定存放在附加段中。

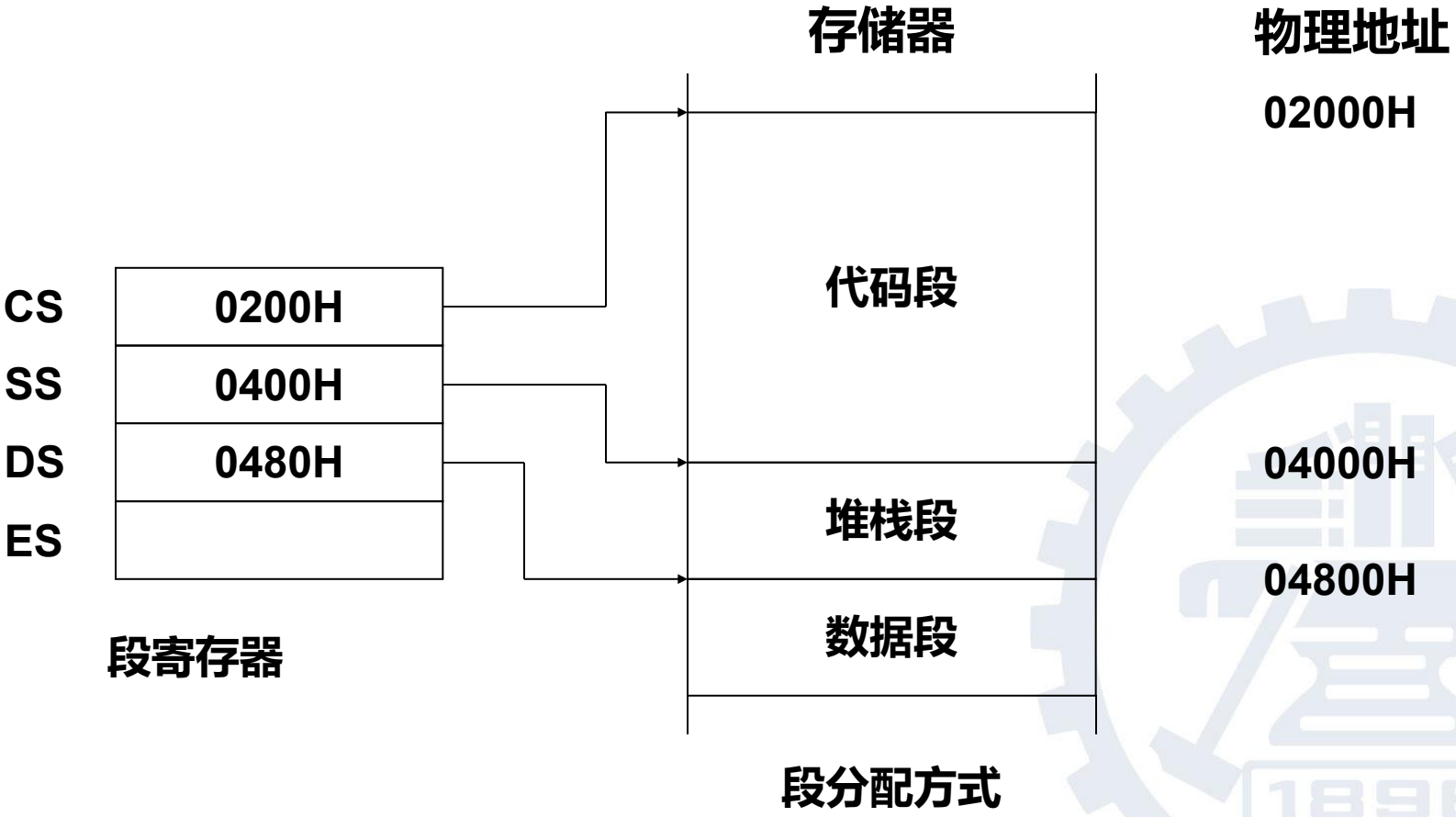
每个段寄存器可以确定一个段的20位起始地址中的高16位，称为段基址

段地址与段寄存器

实模式下，在8086～Pentium微处理机中，代码段的段地址放在CS中；数据段的段地址放在DS中；堆栈段的段地址存放在SS中；附加段的段地址存放在ES中。

将CS,DS,SS,ES称为段寄存器

各段在存储器中的分配是由操作系统负责的。一般情况下，根据需求量分配。注意：只有CS段寄存器是OS分配的，应用程序可读但不可写，其它的段寄存器都可以被修改。



隐含段和偏移寄存器

8086～Pentium微处理机中，段寄存器和偏移寄存器组合有一定规则。

缺省16位段+偏移寻址组合

段	偏　移	主要用途
CS	IP	指令寻址
SS	SP或BP	堆栈寻址
DS	BX,DI,SI或16位数	数据寻址
ES	串指令DI	目标串寻址

注意：当BX/BP与SI/DI组合作为偏移地址时，默认的段寄存器是BX/BP所在段的寄存器，而不是SI/DI所在段的寄存器。

例1：(DS) = 2100H，(BX) = 0500H

物理地址 = 21000H + 0500H = 21500H

例2：(CS) = 3000H，(IP) = 1000H

物理地址 = 30000H + 1000H = 31000H

例3：(SS) = 1000H，(SP) = 0010H

物理地址 = 10000H + 0010H = 10010H

例4：(DS) = 3100H，(SI) = 1000H

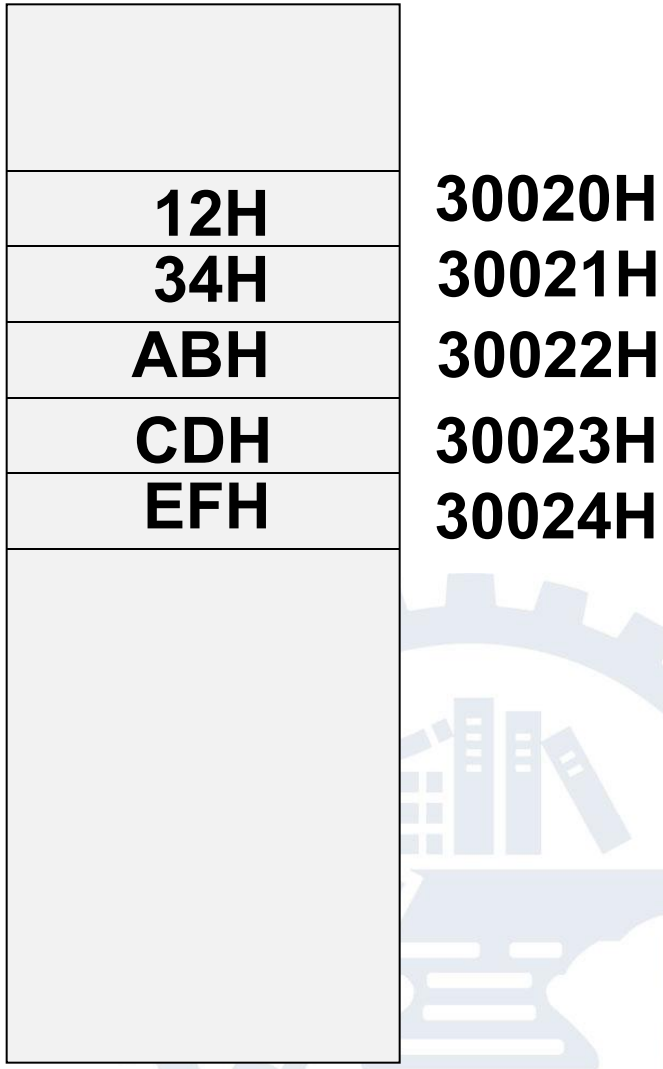
物理地址 = 31000H + 1000H = 32000H



作 业

1、有两个16位字1EE5H和2A3CH分别存放在微机存储器000B0H和000B3H单元中，请用图示表示出它们在存储器里的存储情况。

2、微机存储器里存放的信息如右图示，试读出30022H和30024H字节单元的内容，以及30021H和30022H字单元中的内容。



3、在实模式下，段地址和偏移地址为3017H:000AH的存储单元的物理地址是什么？如果段地址和偏移地址是3015H:002AH和3010H:007AH呢?

4、如果在一个程序开始执行之前（CS）=0A7F0H（如果十六进制数的最高位为字母，则应该在其前加1个0），（IP）=2B40H，试问该程序的第1个字的物理地址是多少?